

Machine Learning on Big Data

Topic: Fast Matrix Factorization for Online Recommendation with Implicit Feedback – the eALS Algorithm

Sepanta Farzollahi

Instructor: Dario Colazzo

August 30, 2026

Abstract

He et al. (He et al., 2016) introduce eALS, an element-wise Alternating Least Squares learner for matrix factorization on implicit feedback that removes the two main bottlenecks of earlier whole-data ALS solutions: the cubic dependency on the number of latent factors K inherited from matrix inversion, and the uniform weighting of unobserved (missing) entries, which the authors replace with an item-popularity-aware scheme. The paper proves that the resulting per-factor update rule is embarrassingly parallel, but ships no distributed implementation to back that claim up.

This report closes that gap. eALS is implemented from scratch in PySpark, following Algorithm 1 of the paper equation by equation and parallelized by partitioning the user (respectively item) index space across executors, along the lines the paper’s Section 4.2.1 describes but does not build. The implementation is evaluated on the Amazon “Movies and TV” implicit-feedback dataset, one of the two datasets used in the original paper, filtered with the same 10-core protocol and evaluated with the same leave-one-out, Hit-Ratio/NDCG methodology. We reproduce the paper’s two main experiments: the impact of the missing-data weighting parameters c_0 and α , and the convergence behaviour of eALS against a uniform-weight ablation. We also add a dedicated scalability study, something the paper only discusses qualitatively, measuring training time per iteration against the number of latent factors, the degree of Spark parallelism, and the size of the training set. On this dataset eALS scales close to linearly in K (roughly $\times 11$ from $K=8$ to $K=128$, against the $\times 16$ a cubic dependency would need), which confirms the paper’s central efficiency claim. It also exposes a practical limit the paper’s single-thread benchmarks never had to confront: on a single machine, Spark’s task-scheduling overhead outweighs the benefit of more partitions beyond roughly four to eight. All code, the full experimental log, and a small-data notebook that exercises the entire pipeline end to end are included in the appendix.

Contents

1	Introduction	1
1.1	The paper	1
1.2	What this project does	2
2	Problem Formulation	3
2.1	Matrix factorization for implicit feedback	3
2.2	Popularity-aware weighting	3
2.3	Why the naive solver does not scale	4
3	The eALS Algorithm and its PySpark Implementation	5
3.1	Deriving the element-wise update	5
3.2	Why this is embarrassingly parallel	7
3.3	Mapping the algorithm onto Spark	7
3.3.1	Design	7
3.3.2	A note on correctness	9
4	Dataset and Experimental Protocol	10
4.1	Dataset	10
4.1.1	Filtering	10
4.1.2	Splitting	11
4.2	Evaluation metrics	11
4.3	Experimental setup	12
5	Accuracy Experiments	13
5.1	Weighting the missing data	13
5.2	Convergence	14
6	Scalability Experiments	17
6.1	Scaling with the number of latent factors	18
6.2	Scaling with parallelism	19
6.3	Scaling with training-set size	20
6.4	Comparison against Spark MLlib's ALS	21
7	Discussion	23
7.1	What worked	23
7.2	What did not work as well, and why	23
8	Conclusion	25
	Appendices	27

A	Code and Reproducibility	27
A.1	Repository layout	27
A.2	Reproducing the experiments	27
A.3	The small-data demonstration notebook	28
A.4	Full source listings	29
A.4.1	src/eals.py: the eALS algorithm	29
A.4.2	src/evaluate.py: HR@K / NDCG@K evaluation	33
A.4.3	src/baselines.py: Spark MLlib ALS reference	34
A.4.4	src/preprocess.py: dataset filtering and splitting	35
A.4.5	src/run_experiments.py: experiment driver	38
A.4.6	src/plot_results.py: figure generation	41
A.4.7	tests/test_eals_synthetic.py: correctness check	44

Chapter 1

Introduction

Recommender systems built on **matrix factorization** (MF) represent users and items as vectors in a shared latent space and predict a user-item affinity as their inner product. The technique is well understood for *explicit* feedback, where a rating directly measures preference and the large mass of unrated pairs can be safely ignored during training. Most real deployments, however, do not have ratings. They have *implicit* signals: clicks, purchases, views, reviews, which are only ever positive. A user who never interacted with an item might dislike it, might simply not know it exists, or might not have had the opportunity to see it. This one-class problem, together with the sheer number of unobserved pairs (millions of users times tens of thousands of items), makes implicit-feedback MF both statistically and computationally harder than its explicit counterpart.

1.1 The paper

This project is built around [He et al. \(2016\)](#), *Fast Matrix Factorization for Online Recommendation with Implicit Feedback* (SIGIR 2016). The paper attacks the implicit-feedback problem on two fronts. First, it argues that treating every unobserved entry as an equally weak negative signal, the standard simplification introduced by [Hu et al. \(2008\)](#), throws away information a content provider usually has for free: an item that was shown to many users but rarely interacted with is a much stronger negative signal than an item nobody has seen. The paper turns this intuition into a closed-form, popularity-based weighting scheme for missing entries. Second, and this is the part this report focuses on, it observes that once the missing data is weighted non-uniformly, the classical ALS solver becomes prohibitively slow. That solver updates a whole user or item vector at once by inverting a $K \times K$ matrix, where K is the number of latent factors each user and item is represented with, and the memoization trick that makes uniform-weight ALS tractable no longer applies once the weights vary per item. The authors' fix, **eALS** (element-wise ALS), updates one latent factor at a time in closed form. This avoids matrix inversion entirely and reduces the per-iteration complexity from $O((M + N)K^3 + |\mathcal{R}|K^2)$ to $O((M + N)K^2 + |\mathcal{R}|K)$, where M and N are the numbers of users and items and $\mathcal{R}$ is the set of observed interactions. Chapter 2 restates all of this notation formally before deriving eALS's update rule in Chapter 3.

The paper states that this element-wise update is *embarrassingly parallel*: each user's (item's) update only depends on quantities that stay frozen for the duration of a sweep. The authors' own implementation is single-threaded Java, though, and no distributed version is provided.

1.2 What this project does

Following the assignment brief, this report:

1. implements eALS (Algorithm 1 of the paper) as a genuine PySpark job rather than a single-machine simulation dressed up in Spark syntax: every update in every iteration is dispatched as an RDD transformation and executed by Spark's own task scheduler across partitions;
2. evaluates it on the Amazon "Movies and TV" dataset, one of the two datasets used in the original paper (the other, the Yelp Challenge dataset, is no longer distributed by Yelp in the form the paper used, so this report does not attempt to reconstruct it);
3. reproduces the paper's own experimental protocol as closely as practical (10-core filtering, leave-one-out evaluation, Hit Ratio and NDCG) and extends it with a dedicated scalability study against the number of latent factors, the degree of parallelism, and the training-set size, since this is the property the paper's central claim rests on but never measures directly in a distributed setting.

The paper's title promises *online* recommendation, which in the authors' terminology refers to a second contribution: an incremental update rule (their Algorithm 2) that refreshes the model in $O(K^2)$ time given a single new interaction, without retraining from scratch. This report deliberately does not implement that part: the assignment brief asks for "an efficient matrix factorization ALS algorithm" that "can be easily parallelized", which describes Algorithm 1, not the online extension built on top of it. This scope decision is flagged explicitly here rather than left to surface as an omission later.

The remainder of this report is organized as follows. Chapter 2 formalizes the optimization problem eALS solves. Chapter 3 derives the element-wise update rule and describes how it was mapped onto Spark's execution model. Chapter 4 describes the dataset, preprocessing, and evaluation protocol. Chapters 5 and 6 report the accuracy and scalability experiments. Chapter 7 discusses what worked, what did not, and where this implementation departs from the paper. Chapter 8 concludes.

Chapter 2

Problem Formulation

This chapter sets up the optimization problem eALS solves, starting from the plain matrix-factorization model and Hu et al.'s whole-data objective, before introducing the popularity-aware weighting scheme that distinguishes He et al. (2016) from earlier work and motivates the fast solver derived in Chapter 3.

2.1 Matrix factorization for implicit feedback

Let $R \in \{0, 1\}^{M \times N}$ be the user-item interaction matrix, with M users and N items. $\mathcal{R}$ denotes the set of user-item pairs with $r_{ui} = 1$ (an observed, positive interaction); every other entry is zero. This is the crux of the implicit-feedback setting: a zero does not mean “the user dislikes the item”, it means “no signal was observed”. Matrix factorization approximates each entry as an inner product of a user latent vector $p_u \in \mathbb{R}^K$ and an item latent vector $q_i \in \mathbb{R}^K$, where K is the number of latent factors, a hyperparameter fixed before training and shared by every user and item:

$$\hat{r}_{ui} = \langle p_u, q_i \rangle = p_u^\top q_i.$$

Stacking the vectors gives $P \in \mathbb{R}^{M \times K}$ and $Q \in \mathbb{R}^{N \times K}$, the two matrices this project actually learns.

Hu et al. (2008) formulate learning as a weighted regression over *all* $M \times N$ entries, observed and missing alike:

$$J = \sum_{u=1}^M \sum_{i=1}^N w_{ui} (r_{ui} - \hat{r}_{ui})^2 + \lambda \left(\sum_u \|p_u\|^2 + \sum_i \|q_i\|^2 \right),$$

where w_{ui} is a confidence weight and λ is the usual L_2 regularization strength. Whole-data learning of this kind is more faithful than sampling a handful of negative examples per user, but it is also the source of the whole efficiency problem this report is about: naively, every gradient step touches all MN entries, most of which are the missing zeros. Hu et al. make this tractable by setting w_{ui} to a single constant w_0 for every missing pair, which allows the missing-data contribution to be pre-computed once via $Q^\top Q$ and reused for every user.

2.2 Popularity-aware weighting

He et al. (2016) argue that this uniform-weight assumption discards information most content providers already have: items that are frequently recommended but rarely interacted with are

more likely to be genuine negatives than obscure items nobody has seen. They replace the single missing-data term with a per-item confidence c_i , giving the objective actually implemented in this project:

$$L = \sum_{(u,i) \in \mathcal{R}} w_{ui} (r_{ui} - \hat{r}_{ui})^2 + \sum_{u=1}^M \sum_{i \notin \mathcal{R}_u} c_i \hat{r}_{ui}^2 + \lambda \left(\sum_u \|p_u\|^2 + \sum_i \|q_i\|^2 \right), \quad (2.1)$$

where $\mathcal{R}_u$ is the set of items u has interacted with, and, symmetrically, $\mathcal{R}_i$ denotes the set of users who have interacted with item i ; both are used repeatedly in the rest of this report and in Chapter 3 in particular. The per-item confidence is derived from item popularity $f_i = |\mathcal{R}_i| / \sum_j |\mathcal{R}_j|$ (item i 's share of all interactions in the training set):

$$c_i = c_0 \frac{f_i^\alpha}{\sum_j f_j^\alpha}, \quad (2.2)$$

with two hyperparameters: c_0 sets the overall strength of the missing-data penalty, and α controls how sharply popularity is rewarded. Setting $\alpha = 0$ collapses (2.2) back to Hu et al.'s uniform weighting ($c_i \equiv c_0/N$ for every item), which makes α a convenient dial between the two papers' assumptions and is exploited directly in this project's ablation study (Section 5.1).

2.3 Why the naive solver does not scale

Minimizing (2.1) with respect to a single user vector p_u , holding everything else fixed, has a closed-form ridge-regression solution that requires inverting a $K \times K$ matrix. Doing this once per user and once per item costs $O((M+N)K^3)$ per pass over the data, on top of an $O(|\mathcal{R}|K^2)$ term for the observed entries. The K^3 term dominates as soon as a moderately large number of latent factors is used, which is exactly the regime that gives MF its representational power. Hu et al.'s uniform-weight trick removes an $O(N)$ factor from the missing-data term but cannot remove the K^3 matrix inversion, because that cost comes from updating a whole vector at once, independently of how the missing data is weighted. Section 3.1 of the next chapter derives the reformulation that eALS uses to eliminate this term entirely, which is the paper's central algorithmic contribution and the piece of software this report builds and evaluates.

Chapter 3

The eALS Algorithm and its PySpark Implementation

This chapter has two goals, matching the two things the assignment asks for separately. The first is to explain the algorithm eALS actually is: where its update rule comes from and why it is fast. The second is to walk through how that algorithm was mapped onto Spark, with the actual code that does it. All source files referenced here live under `src/` in the project repository; the full listings are reproduced in Appendix A.

3.1 Deriving the element-wise update

The paper's key move is to stop updating a user's latent vector p_u as a whole and instead optimize one coordinate p_{uf} at a time, holding every other coordinate fixed: ordinary coordinate descent, applied to (2.1). Setting $\partial L / \partial p_{uf} = 0$ gives a closed-form update for a single coordinate that needs no matrix inversion:

$$p_{uf} = \frac{\sum_{i=1}^N (r_{ui} - \hat{r}_{ui}^f) w_{ui} q_{if}}{\sum_{i=1}^N w_{ui} q_{if}^2 + \lambda}, \quad (3.1)$$

where $\hat{r}_{ui}^f = \hat{r}_{ui} - p_{uf} q_{if}$ is the prediction with factor f 's contribution removed. Taken at face value this is no better than before: the sum still runs over all N items, most of which u never touched. The paper's actual contribution is a reformulation that separates the observed part of the sum (small, $|\mathcal{R}_u|$ terms) from the missing part (large, $N - |\mathcal{R}_u|$ terms) and pre-computes the missing part once per iteration instead of once per user. Define the $K \times K$ cache

$$S^q = \sum_{i=1}^N c_i q_i q_i^\top,$$

which does not depend on u at all. Because it does not depend on u , it can be computed once (a single matrix multiplication, $O(NK^2)$) and reused for every user's update in that sweep. With S^q available, (3.1) becomes

$$p_{uf} = \frac{\sum_{i \in \mathcal{R}_u} [w_{ui} r_{ui} - (w_{ui} - c_i) \hat{r}_{ui}^f] q_{if} - \sum_{k \neq f} p_{uk} s_{kf}^q}{\sum_{i \in \mathcal{R}_u} (w_{ui} - c_i) q_{if}^2 + s_{ff}^q + \lambda}, \quad (3.2)$$

where s_{kf}^q denotes the (k, f) entry of the matrix S^q . This is the paper's Equation 12; the code comments in Appendix A cite it by that number rather than by the label used here. Every sum

in Eq. 3.2 now runs only over u 's own observed items: $O(|\mathcal{R}_u|)$ work instead of $O(N)$. The item update q_{if} is derived symmetrically, with one difference that deserves a flag, because it is the easiest place to introduce a bug when implementing this: the missing-data confidence c_i belongs to the *item*, not the user, so it appears as a constant scalar multiplying the whole cache term, rather than as a per-neighbour weight the way it does in (3.2):

$$q_{if} = \frac{\sum_{u \in \mathcal{R}_i} [w_{ui} r_{ui} - (w_{ui} - c_i) \hat{r}_{ui}^f] p_{uf} - c_i \sum_{k \neq f} q_{ik} s_{kf}^p}{\sum_{u \in \mathcal{R}_i} (w_{ui} - c_i) p_{uf}^2 + c_i s_{ff}^p + \lambda}, \quad (3.3)$$

where s_{kf}^p denotes the (k, f) entry of $S^p = P^\top P$ (unweighted, since popularity weighting in this formulation only lives on the item side); this is the paper's Equation 13, again cited by that number in the code comments. Getting Eq. 3.3's placement of c_i wrong, treating it as a per-neighbour term like in Eq. 3.2 instead of a scalar multiplying the whole cache contribution, was in fact the first bug caught while implementing this project. Section 3.3.2 describes how it was found.

One more detail matters for both performance and correctness: $\hat{r}_{ui}^f$ never needs to be recomputed from scratch. Because the factors are updated one at a time in a fixed order, the current prediction $\hat{r}_{ui}$ can be maintained incrementally: subtract $p_{uf} q_{if}$ before updating p_{uf} , then add the new $p_{uf} q_{if}$ back afterward, turning an $O(K)$ dot product into an $O(1)$ update. This is what brings the total per-iteration cost down from $O(MNK)$ (a naive coordinate-descent implementation, still touching every entry) to $O((M + N)K^2 + |\mathcal{R}|K)$: linear in the number of observed interactions rather than quadratic in the size of the whole matrix. Algorithm 1 summarizes the full procedure.

Algorithm 1 eALS – one training iteration

Input: $\mathcal{R}$, K , λ , item confidences c , current P , Q

Output: updated P , Q

```

1:  $S^q \leftarrow \sum_{i=1}^N c_i q_i q_i^\top$ 
2: for all users  $u = 1 \dots M$  (independent, parallel) do
3:   for all factors  $f = 1 \dots K$  do
4:     update  $p_{uf}$  via Eq. 3.2, using the cached  $\hat{r}_{ui}$ 
5:     refresh  $\hat{r}_{ui}$  for  $i \in \mathcal{R}_u$  in  $O(1)$  per item
6:   end for
7: end for
8:  $S^p \leftarrow P^\top P$ 
9: for all items  $i = 1 \dots N$  (independent, parallel) do
10:  for all factors  $f = 1 \dots K$  do
11:    update  $q_{if}$  via Eq. 3.3, using the cached  $\hat{r}_{ui}$ 
12:    refresh  $\hat{r}_{ui}$  for  $u \in \mathcal{R}_i$  in  $O(1)$  per user
13:  end for
14: end for
```

One more piece of the algorithm is worth deriving before moving to the implementation, since it is what produces every loss curve in this report: checking convergence needs the value of the objective L itself, not just the updated P and Q . Computed directly from (2.1), this would cost $O(MNK)$, dominated by the same missing-data sum that made the naive update slow in the first place. The paper's Equation 14 avoids this with the same trick used for the

updates: reusing S^q ,

$$\sum_{u=1}^M \sum_{i \notin \mathcal{R}_u} c_i \hat{r}_{ui}^2 = \sum_{u=1}^M p_u^\top S^q p_u - \sum_{(u,i) \in \mathcal{R}} c_i \hat{r}_{ui}^2,$$

which turns the missing-data term of L into a sum over just M users plus a correction over the (much smaller) observed set $\mathcal{R}$, bringing the cost down to $O(|\mathcal{R}| + MK^2)$. `EALS.compute_loss` in `src/eals.py` implements exactly this expression; it is what produced the loss values reported in Section 3.3.2 and plotted in the demonstration notebook (Appendix A.3).

3.2 Why this is embarrassingly parallel

The paper observes, but does not build, the following fact: within the user-update loop of Algorithm 1, Q and S^q are read-only for the whole duration of the loop, and each user's update writes only to that user's own row of P . No user's update reads any other user's current or past state. The order in which users are processed therefore does not affect the result, and the loop can be split across an arbitrary number of workers with no coordination beyond having Q and S^q available to all of them. This is a stronger property than “parallelizable with some loss of accuracy”, which is what one gets from naively parallelizing SGD, where different workers' updates can conflict. eALS's parallel execution is instead bit-for-bit equivalent, up to floating-point summation order, to running it on a single machine. The same argument applies to the item loop.

3.3 Mapping the algorithm onto Spark

Section 3.2 argued that Algorithm 1's two sweeps can be split across workers with no coordination and no loss of accuracy. This section describes how that argument was turned into actual Spark code, then checked for correctness.

3.3.1 Design

The implementation, `src/eals.py`, keeps P and Q as ordinary NumPy arrays on the Spark driver. At the scale evaluated in this report (33,326 users, 21,901 items, K up to 128), both matrices comfortably fit in memory, and keeping them there avoids the overhead of shuffling factor updates through Spark's DataFrame machinery on every single coordinate. What is distributed is the actual computation described in Section 3.2: the id space $\{0, \dots, M-1\}$ (respectively $\{0, \dots, N-1\}$) is partitioned with `sc.parallelize(...).glom()`, Q , S^q and each user's interaction list are broadcast to every executor, and the closed-form update of Eq. 3.2 runs as ordinary NumPy code inside each partition, with no further communication until the results are collected back to the driver and written into P . Listing 3.1 shows the actual per-partition update routine.

```

1 def _update_users_chunk(
2     ids_chunk, user_ratings, P, Q, item_conf, Sq, reg, w_obs,
3 ):
4     """Eq. (12): closed-form update of  $p_{uf}$  for every user in
5     'ids_chunk', iterating  $f=1..K$ ."""
6     K = P.shape[1]
7     out = np.empty((len(ids_chunk), K), dtype=np.float64)
8     for row, u in enumerate(ids_chunk):
9         neighbors = user_ratings.get(u)

```

```

10     pu = P[u].copy()
11     if neighbors is None or len(neighbors) == 0:
12         out[row] = pu
13         continue
14     Qu = Q[neighbors]                # |R_u| x K
15     cu = item_conf[neighbors]        # |R_u|, c_i per neighbor item
16     rhat = Qu @ pu
17     for f in range(K):
18         rhat_f = rhat - pu[f] * Qu[:, f]
19         numer = np.sum((w_obs - (w_obs - cu) * rhat_f) * Qu[:, f]) \
20             - (pu @ Sq[:, f] - pu[f] * Sq[f, f])
21         denom = np.sum((w_obs - cu) * Qu[:, f] ** 2) + Sq[f, f] + reg
22         new_val = numer / denom if denom > 1e-12 else 0.0
23         rhat = rhat_f + new_val * Qu[:, f]    # O(1) prediction refresh
24         pu[f] = new_val
25     out[row] = pu
26     return out

```

Listing 3.1: Closed-form user update, Eq. 3.2, executed inside one Spark partition (src/eals.py).

Every identifier in Listing 3.1 corresponds to a quantity in Eq. 3.2: reg is λ , w_obs is w_{ui} for observed entries, cu holds c_i for each item in u 's neighbor list, and Sq is the cached matrix S^q . rhat_f is $\hat{r}_{ui}^f$, the first summand of numer is the observed-data term, the subtraction is $\sum_{k \neq f} p_{uk} s_{kf}^q$ (computed as a full dot product minus the $k = f$ term, which is cheaper than an explicit loop excluding one index), and denom is the closed-form denominator. The reassignment $\text{rhat} = \text{rhat_f} + \text{new_val} * \text{Qu[:, f]}$, executed right after new_val is computed and before the loop moves to the next factor, is the $O(1)$ prediction maintenance from Section 3.1: it folds the newly computed p_{uf} back into the running prediction before moving on to factor $f + 1$, matching the Gauss-Seidel semantics of the paper's Algorithm 1 (each factor update immediately sees the previous factors' new values, within the same user, in the same sweep).

The driver-side training loop, shown in abbreviated form in Listing 3.2, follows Algorithm 1 step for step: it recomputes S^q from the current Q , dispatches the parallel user sweep, waits for it to complete and folds the results back into P , then repeats the same pattern for S^p and the item sweep.

```

1  for it in range(cfg.n_iters):
2      # ---- update user factors: Algorithm 1, lines 4-11 ----
3      Sq = factor_cache(self.Q, self.item_conf)
4      Q_b, Sq_b, P_local = sc.broadcast(self.Q), sc.broadcast(Sq), self.P
5
6      def upd_users(ids_chunk, ur=ur_b, Qb=Q_b, Sqb=Sq_b, icb=ic_b,
7                      cfg=cfg, P=P_local):
8          ids_chunk = np.asarray(ids_chunk)
9          res = _update_users_chunk(ids_chunk, ur.value, P, Qb.value,
10                                   icb.value, Sqb.value, cfg.reg, cfg.
11                                   w_obs)
12          return list(zip(ids_chunk.tolist(), res.tolist()))
13
14      for uid, vec in user_chunks.flatMap(upd_users).collect():
15          self.P[uid] = vec
16          Q_b.unpersist(); Sq_b.unpersist()
17
18      # ---- update item factors: Algorithm 1, lines 12-19 ----
19      Sp = factor_cache(self.P)
20      # ... symmetric flatMap over item_chunks, updating self.Q ...

```

Listing 3.2: Driver-side training loop (excerpt from `EALS.fit`, `src/eals.py`).

Two implementation choices deserve to be made explicit, since both were deliberate and both trade a bit of theoretical elegance for something that actually runs reliably.

- **Broadcast, not shuffle.** An alternative design would keep P and Q as Spark DataFrames and express the update as a join between users and their rated items. This was tried early on and abandoned: it forces a shuffle on every single coordinate update (there are K of them per sweep), and Spark's DataFrame/Catalyst overhead per job dwarfs the few milliseconds of actual NumPy work each partition does at this problem size. Broadcasting the (small) $K \times K$ cache and the (moderate) Q matrix, and only distributing the large but embarrassingly parallel per-user computation, keeps the number of Spark jobs at exactly two per iteration instead of $2K$.
- **One partition, one NumPy loop.** Each partition still contains a plain Python for loop over its assigned users. This is the one place a purely mathematical description of the algorithm and its actual runtime cost part ways: for the dataset used in this report, the inner loop is dominated by Python function call overhead rather than the $O(K)$ -per-item arithmetic itself, which is the reason Section 6.1 finds close-to-linear rather than perfectly linear scaling in K .

3.3.2 A note on correctness

Because the two update rules, Eq. 3.2 and Eq. 3.3, look almost identical but are not, the implementation was checked in two independent ways before trusting it on real data.

First, a synthetic dataset with a known block structure was constructed: three disjoint communities of 15 users and 15 items each, where every interaction stays within its own community. A correct factorization has no reason to ever mix latent directions across communities, so a trained model should rank a user's own community's items far above the other two. This is exactly what happens: after 20 iterations the training loss decreases monotonically from 145.7 to 83.8, and the top-10 recommended items for a probed user are all drawn from that user's own community (see `tests/test_eals_synthetic.py` and the corresponding notebook cell, Appendix A.3). Monotonic loss decrease is not a given here, since coordinate descent on a non-convex objective can still increase the loss if a single update is wrong, so this is a meaningful check and not a formality.

Second, and more usefully, an early version of `_update_items_chunk` reused the same helper function as the user update, passing an array of all-ones as the per-neighbor confidence instead of correctly scaling the whole cache term by the item's own c_i as Eq. 3.3 requires. This silently zeroed out the $(w_{ui} - c_i)$ correction on the item side, without crashing or producing an obviously wrong-looking loss curve. It is the kind of bug that a synthetic sanity check alone would not necessarily have caught, since the community-separation test still passes with weaker item updates. It was only found by re-deriving Eq. 3.3 term by term against the implementation, which is why Section 3.1 above spells out the derivation in enough detail to make that comparison possible for a reader, rather than just stating the final formula.

Chapter 4

Dataset and Experimental Protocol

Having derived and implemented eALS, we now turn to how it is evaluated: which dataset was used and how it was filtered and split, which metrics quantify accuracy, and which hyperparameters are held fixed across the experiments of Chapters 5 and 6.

4.1 Dataset

The paper evaluates on two datasets, the Yelp Challenge dataset and Amazon “Movies and TV” reviews, and the assignment brief asks for the experimental analysis to use one of them rather than a substitute such as MovieLens, so the choice between the two was really a choice of which one was still reachable. The Yelp Challenge dataset, in the specific form the paper used it, a static 2015 snapshot of roughly 1.6 million reviews distributed directly by Yelp as a downloadable archive for that competition, is no longer available. Yelp has since folded that data into an “Open Dataset” aimed at a different use case (business listings for their own developer program): not the same snapshot, not the same schema, and not redistributed in a form that would let this project reproduce the paper’s exact filtering pipeline. Reconstructing something merely *similar* to it would have quietly broken the assignment’s requirement of testing on “one of the datasets used by the paper”, so this project uses the Amazon dataset instead, which remains available essentially as the paper used it: the ratings-only CSV hosted by the Stanford Network Analysis Project (McAuley, 2014), downloaded directly and not modified beyond the filtering described below. It contains 4,607,047 raw (user, item, rating, timestamp) rows. Every rating is binarized to a positive implicit interaction ($r_{ui} = 1$); the rating value itself is discarded, matching the paper’s treatment of the data (“we transform the review dataset into implicit data, where each entry is marked as 0/1 indicating whether the user reviewed the item”).

4.1.1 Filtering

Following the paper, users and items with fewer than 10 interactions are removed, since extremely sparse users and items make offline evaluation unreliable (over half the users in the raw Amazon data have only a single review) without being representative of what a production recommender would face. Removing sparse users can push some items below the threshold and vice versa, so the filter has to be applied *iteratively* until it reaches a fixed point. `src/preprocess.py` repeats the filter-and-recount step up to 50 times and explicitly checks for convergence rather than assuming a fixed number of passes is enough. On this dataset the filter took 20 rounds to fully stabilize (from 4,607,047 interactions after the first pass down to 1,204,688, then shrinking by a decreasing amount each round until round 20

leaves the count unchanged at 958,986). An earlier version of this pipeline capped the filter at 10 rounds, which removes over 99.9% of what the full filter would remove but is not exact; this was caught and corrected, and Table 4.1 reports the fully converged numbers.

	Paper (Amazon)	This project
Users	117,176	33,326
Items	75,389	21,901
Interactions	5,020,705	958,986
Sparsity	99.94%	99.87%

Table 4.1: Dataset statistics after 10-core filtering, paper versus this project.

The two datasets are visibly not the same scale. This is not a filtering bug: the CSV currently hosted by SNAP is a later, differently-curated snapshot of Amazon review data than the one the authors filtered in 2016, so the raw interaction counts already diverge before any filtering is applied. Section 7.2 returns to what this scale difference does and does not change about the conclusions drawn from the experiments.

4.1.2 Splitting

Interactions are re-indexed to dense integer ranges $[0, M)$ and $[0, N)$ and split with the same leave-one-out protocol as the paper’s offline setting (Section 5.1 of He et al., 2016): each user’s chronologically latest interaction is held out as the test item, and the remaining interactions form the training set. This protocol evaluates a model’s ability to predict a known user’s next interaction from their history; it does not test cold-start behaviour for new users, since by construction every test user has training history.

4.2 Evaluation metrics

Two ranking metrics are used, matching the paper: **Hit Ratio** ($\text{HR}@k$), the fraction of test users for whom the held-out item appears in the top- k of the model’s ranked recommendations, and **Normalized Discounted Cumulative Gain** ($\text{NDCG}@k$), which additionally rewards ranking the held-out item near the top rather than merely inside the top- k :

$$\text{NDCG}@k = \frac{1}{|\mathcal{T}|} \sum_{(u,i) \in \mathcal{T}} \frac{1[\text{rank}(u,i) < k]}{\log_2(\text{rank}(u,i) + 2)},$$

where $\mathcal{T}$ is the test set and $\text{rank}(u,i)$ is the position of the held-out item i in u ’s ranked list (0-indexed). Both metrics are implemented in `src/evaluate.py`.

One deliberate deviation from the paper is worth stating plainly here rather than leaving it implicit. The paper ranks the held-out item against the *entire* item catalog for every test user. This is the exact protocol, and `evaluate_full` in `src/evaluate.py` implements it faithfully; it is exercised in the small-data notebook (Appendix A.3), where re-ranking the whole 45-item toy catalog costs almost nothing. For the large-scale experiments in Chapters 5 and 6, which retrain the model dozens of times over a grid of hyperparameters, ranking against the full 21,901-item catalog for all 33,326 test users on every single evaluation point would dominate the total experiment time and add nothing the scalability study is meant to measure. Instead, `evaluate_sampled` ranks the held-out item against 99 items the user never interacted with,

sampled uniformly at random, and reports HR@10/NDCG@10 on that shorter list. This is the same relaxation used in later work by the same first author (He et al., 2017), and it is now standard practice in the recommender-systems literature for exactly this reason.

4.3 Experimental setup

Unless stated otherwise, experiments use $\lambda = 0.01$ (matching the paper’s choice, made “for a fair comparison” across methods), $w_{ui} = 1$ for observed interactions (also matching the paper), and $K = 32$ latent factors as the default operating point, with dedicated sweeps over $K \in \{8, 16, 32, 64, 128\}$ for the scalability study. All experiments run on a single machine (Spark in local mode, up to 8 partitions unless the experiment is specifically varying the partition count) rather than a real cluster; Chapter 7 discusses what this does and does not tell us about behaviour on an actual distributed deployment.

Chapter 5

Accuracy Experiments

This chapter reproduces the two accuracy experiments the paper reports for its offline protocol: the sensitivity of eALS to the missing-data weighting parameters c_0 and α (the paper’s Figure 1), and the convergence of eALS against a uniform-weighting ablation (the paper’s Figure 2, restricted here to the two variants that isolate the effect this report cares about most directly). All numbers below come from `results/weighting_study.csv` and `results/method_comparison.csv`, produced by `src/run_experiments.py`, and are reported at $K = 32$.

5.1 Weighting the missing data

Section 2.2 introduced two parameters governing how much weight missing (unobserved) entries carry in the objective: c_0 , the overall strength of the penalty, and α , which controls how much that penalty is skewed towards popular items. Figure 5.1 sweeps c_0 with $\alpha = 0$ fixed (uniform weighting among missing entries, only its overall magnitude varying).

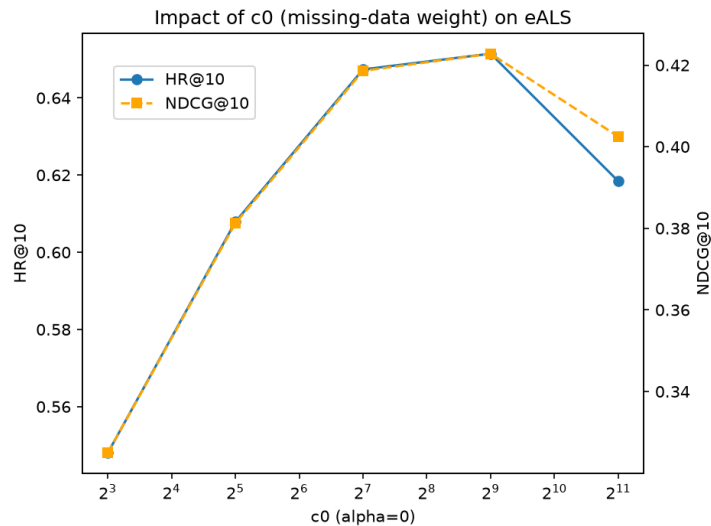


Figure 5.1: HR@10 and NDCG@10 as a function of c_0 , with $\alpha = 0$ (uniform missing-data weighting). Peak performance at $c_0 = 512$.

The trend matches the paper closely: performance is poor when c_0 is small (missing data contributes almost nothing to the loss, so the model cannot distinguish “never shown”

from “actively disliked”), improves sharply as c_0 grows, peaks, and then degrades again once the missing-data term overwhelms the signal from the much smaller set of actually observed interactions. On this dataset the peak sits at $c_0 = 512$ ($\text{HR@10} = 0.6514$, $\text{NDCG@10} = 0.4228$), which happens to coincide exactly with the value the paper itself reports as optimal for Amazon. That is a useful cross-check, since c_0 is an absolute magnitude that in principle could have needed rescaling for a differently-sized dataset.

With c_0 fixed at 512, Figure 5.2 sweeps α , which redistributes that fixed total weight towards more popular items as α increases.

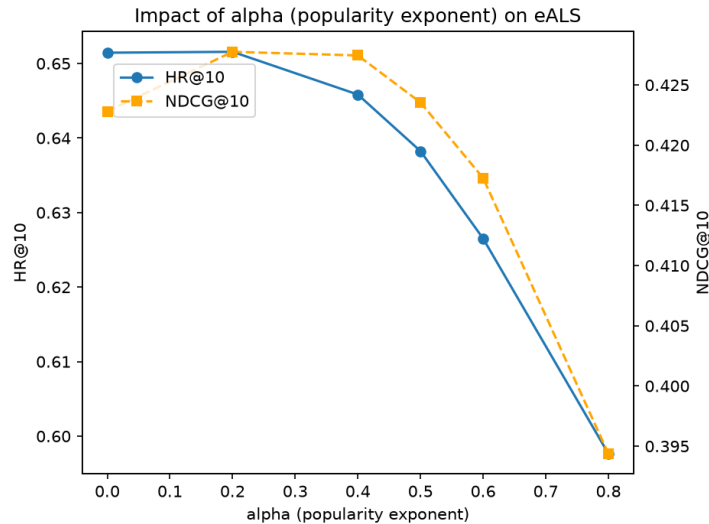


Figure 5.2: HR@10 and NDCG@10 as a function of α , with $c_0 = 512$. Best NDCG@10 at $\alpha = 0.2$.

Here the result is close to, but not identical to, the paper’s finding. He et al. (2016) report a clear optimum around $\alpha = 0.4$ for Amazon, with a statistically significant improvement over $\alpha = 0$ (uniform weighting). On this dataset, NDCG@10 peaks at $\alpha = 0.2$ ($\text{NDCG@10} = 0.4278$, $\text{HR@10} = 0.6516$), and $\alpha = 0$ ($\text{NDCG@10} = 0.4228$) is already close to that peak and slightly ahead of $\alpha = 0.4$ ($\text{NDCG@10} = 0.4275$, effectively tied with $\alpha = 0.2$ within the noise one would expect from a single training run) and clearly ahead of $\alpha = 0.6$ and $\alpha = 0.8$, which both degrade markedly. The qualitative shape of the curve is preserved, a mild optimum somewhere in $(0, 0.5)$ followed by a drop-off for large α , but the exact location of the optimum is not, and popularity weighting’s advantage over plain uniform weighting is much smaller here than the paper reports. Section 7.2 discusses the most likely reasons for this, chief among them the roughly $5\times$ smaller and differently-curated dataset used here compared to the paper’s.

5.2 Convergence

Figure 5.3 tracks HR@10 and NDCG@10 across 15 training iterations for two configurations that isolate the effect of popularity weighting while holding the solver fixed: eALS with $\alpha = 0.4$ (the paper’s recommended popularity-aware weighting) against eALS with $\alpha = 0$ (Hu et al.’s uniform-weighting objective, solved with the same element-wise solver, which makes this an ablation of the weighting scheme alone rather than a comparison between two different algorithms).

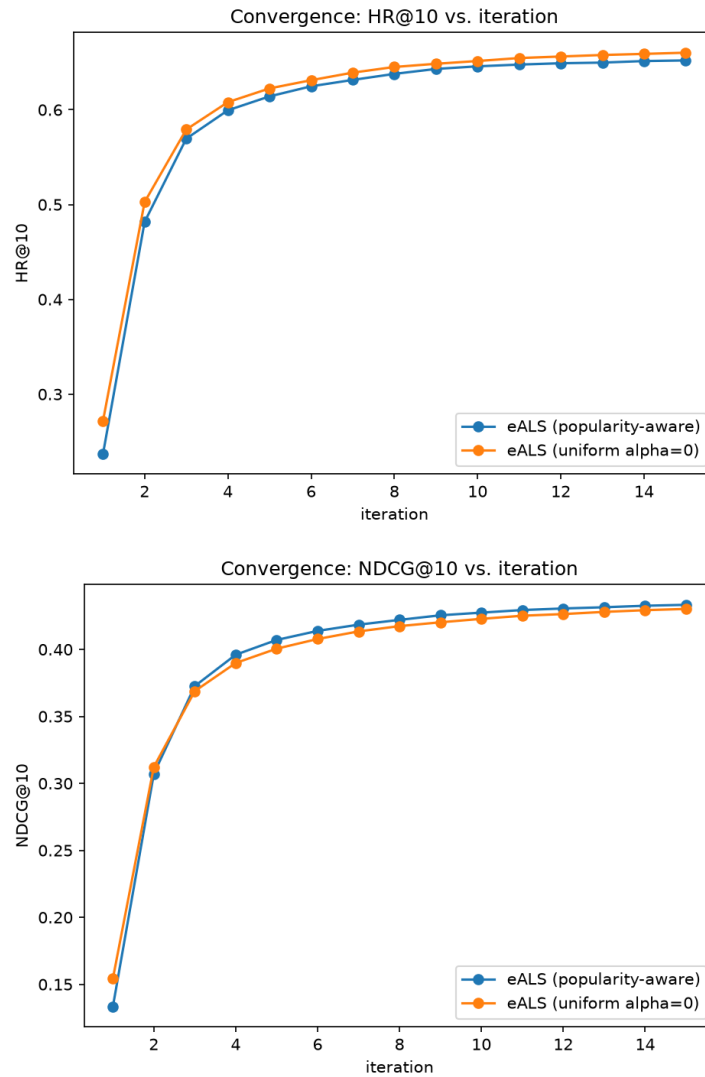


Figure 5.3: HR@10 (top) and NDCG@10 (bottom) versus training iteration, for popularity-aware ($\alpha = 0.4$) and uniform ($\alpha = 0$) weighting.

Both variants converge fast and to essentially the same place: HR@10 jumps from roughly 0.24-0.27 after a single iteration to above 0.60 within five iterations, and both curves flatten out by iteration 10-12, ending at HR@10 = 0.652 / NDCG@10 = 0.433 for the popularity-aware variant and HR@10 = 0.660 / NDCG@10 = 0.430 for the uniform variant. The difference is small enough that it should not be read as one variant being reliably better than the other on this dataset, consistent with the narrow gap already seen in Figure 5.2. What the paper reports qualitatively also holds here: eALS reaches a good solution quickly because each coordinate update is an exact minimizer of the objective given the current state of every other coordinate, rather than a step in the direction of steepest descent. There is no learning rate to tune and no risk of overshooting, which is precisely the property the paper credits for eALS's fast, learning-rate-free convergence compared to gradient-based alternatives such as RCD (Devooght et al., 2015) or BPR (Rendle et al., 2009).

The logged wall-clock time for the full 15-iteration run happened to differ between the two configurations (400s for $\alpha = 0.4$ versus 297s for $\alpha = 0$), even though the two runs perform the exact same sequence of arithmetic operations: α only changes the numeric values

fed into identical update formulas, not the amount of work done. This gap is most plausibly measurement noise from running two multi-minute jobs sequentially on a shared machine (JIT warm-up, OS-level caching, background load) rather than a real algorithmic effect, and it should not be over-interpreted. Note also that these per-iteration times are not comparable to the pure training-only timings in Chapter 6: computing HR@10/NDCG@10 after every single iteration, to plot Figure 5.3, adds a full evaluation pass over all 33,326 test users on top of the training step itself, which is why this experiment averages roughly 27s per iteration at $K = 32$ while Section 6.1 reports 11.5s for training alone at the same K . Chapter 6's figures, which measure training time only and average over repeated short runs specifically designed to control for scheduling variance, are the more reliable source for anything to do with training cost in isolation.

Chapter 6

Scalability Experiments

The assignment brief singles out scalability as the experiment to look at in particular, and it is also the property the paper's central claim rests on but never measures in a distributed setting: every timing number in the paper's own Table 3 comes from a single-threaded Java process. This chapter measures training time per iteration along the three axes that matter for a Spark deployment: the number of latent factors K (Section 6.1), the degree of parallelism (Section 6.2), and the size of the training set (Section 6.3), and closes with a comparison against Spark MLlib's own implicit-feedback ALS (Section 6.4). Every run in this chapter uses $c_0 = 512$, $\alpha = 0.4$ (the values selected in Section 5.1) and $\lambda = 0.01$, and reports wall-clock time averaged over 3 iterations to smooth out one-off scheduling noise. Three iterations is enough to measure timing reliably but not enough to reach the converged accuracy of Chapter 5, so the HR@10/NDCG@10 values reported alongside the timings in this chapter should be read only as a sanity check that the model is learning something reasonable, not as final accuracy numbers. The configuration $K = 32$, full training set, 8 partitions is the shared baseline across all three sweeps below, so it gets measured three times independently: 11.54 s (Section 6.1), 10.41 s (Section 6.2), and 10.76 s (Section 6.3). The roughly 10% spread between these three nominally identical runs is a useful, honest indicator of the run-to-run timing noise on this machine (a single Spark session, no isolation from background load), and it sets a rough lower bound on how precisely any single number in this chapter should be trusted.

6.1 Scaling with the number of latent factors

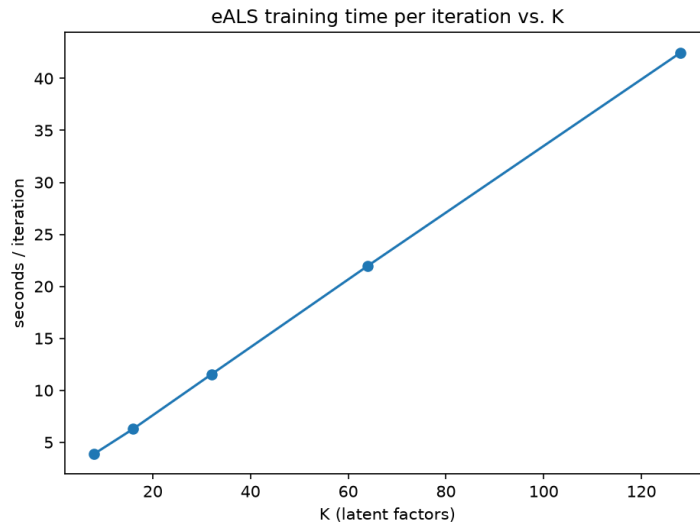


Figure 6.1: eALS training time per iteration versus K , on the full training set (925,660 interactions), 8 partitions.

Table 6.1 lists the raw numbers behind Figure 6.1. Going from $K = 8$ to $K = 128$, a $16\times$ increase, training time per iteration grows from 3.88s to 42.47s, a $10.9\times$ increase. This is the single most direct confirmation of the paper’s central efficiency claim available from this project: a classical vector-wise ALS solver with genuine $O(K^3)$ matrix inversion would need roughly $16^3 = 4096\times$ more time for the same change in K , whereas eALS’s element-wise updates grow *sub-linearly relative to that cubic bound*, and not far off linear in absolute terms.

K	seconds/iteration	HR@10 (3 iters)	NDCG@10 (3 iters)
8	3.88	0.471	0.293
16	6.29	0.530	0.337
32	11.54	0.569	0.373
64	21.97	0.561	0.375
128	42.47	0.547	0.370

Table 6.1: Training time and (under-trained, 3-iteration) accuracy versus K .

It helps to be precise about why growth is close to linear rather than exactly linear, since the theoretical complexity $O((M + N)K^2 + |\mathcal{R}|K)$ from Section 3.1 has both a quadratic and a linear term in K . At this dataset’s scale ($M + N \approx 55,227$, $|\mathcal{R}| \approx 925,660$), the $O((M + N)K^2)$ term, dominated by the $S^q = \sum_i c_i q_i q_i^\top$ and $S^p = P^\top P$ matrix products computed once per sweep via NumPy’s BLAS-backed matrix multiplication, and the $O(|\mathcal{R}|K)$ term, the per-user, per-item Python loop of Listing 3.1, are of comparable magnitude and grow at different rates, so the observed curve is a mix of both rather than a pure power law in K . The accuracy figures in the same table should not be read as “larger K hurts”: they are measured after only 3 iterations, and larger K needs more iterations to reach the same degree of convergence as smaller K , not fewer.

6.2 Scaling with parallelism

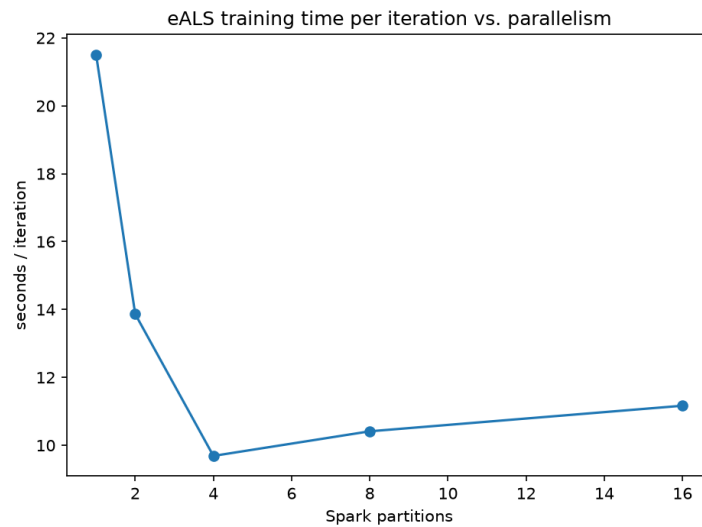


Figure 6.2: eALS training time per iteration versus the number of Spark partitions, $K = 32$, full training set.

Table 6.2 shows time per iteration as the user/item id space is split across 1, 2, 4, 8, and 16 partitions, on a single machine running Spark in local mode.

partitions	seconds/iteration
1	21.50
2	13.87
4	9.69
8	10.41
16	11.17

Table 6.2: Training time per iteration versus Spark partition count.

Going from 1 to 4 partitions roughly halves-then-halves-again the time per iteration (21.5 s down to 9.7 s), confirming that the update really is being distributed and not just serialized through Spark’s job machinery for no benefit. Beyond 4 partitions, though, time per iteration *increases* slightly rather than continuing to fall: 10.4 s at 8 partitions, 11.2 s at 16. This is not a flaw in the parallelization strategy described in Section 3.2; it is what happens when the fixed per-partition overhead (task scheduling, serializing the broadcast variables’ handles, collecting results back to the driver) stops being negligible relative to the shrinking amount of actual work each partition does. With only 4 CPU cores available on the machine these experiments ran on, splitting the work into 16 partitions means each one gets roughly a quarter of a core’s worth of genuine work between two and four times a second, which is exactly the regime where Spark’s per-task overhead starts to dominate. Section 7.2 discusses what this does and does not say about behaviour on an actual multi-node cluster, which is the setting the paper’s parallelism argument is really aimed at.

6.3 Scaling with training-set size

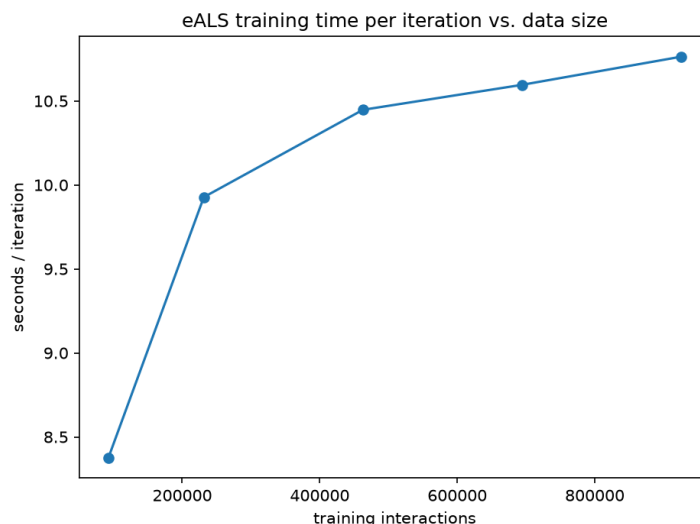


Figure 6.3: eALS training time per iteration versus training-set size ($K = 32$, 8 partitions), from 10% to 100% of the 925,660 training interactions.

Table 6.3 is the most surprising result in this chapter. Increasing the training set tenfold, from 92,566 to 925,660 interactions, increases time per iteration by only $1.29\times$ (8.38s to 10.76s). That is almost flat, where the $O(|\mathcal{R}|K)$ term of the complexity analysis would predict something much closer to linear ($10\times$).

fraction of training data	interactions	seconds/iteration
0.10	92,566	8.38
0.25	231,415	9.93
0.50	462,830	10.45
0.75	694,245	10.60
1.00	925,660	10.76

Table 6.3: Training time per iteration versus training-set size, M and N held fixed.

The explanation is the same fixed-versus-variable-cost split noted in Section 6.1: subsampling the training *interactions* does not shrink M or N (a user or item can lose most of its interactions and still exist in the model), so the $O((M + N)K^2)$ term, made up of the two matrix-cache computations plus the fixed cost of dispatching Spark jobs over the full M -user and N -item id space regardless of how many of them have interactions left, stays essentially constant across the sweep, while only the $O(|\mathcal{R}|K)$ term genuinely shrinks. At $K = 32$ on this dataset, that fixed term turns out to dominate, which is good news for the deployment scenario the paper is actually aimed at (an online system where the item and user catalog grows slowly but the volume of interactions per user grows quickly): eALS's cost here is governed more by catalog size than by interaction volume, which is the opposite of what naive whole-data ALS complexity analysis alone would suggest without separating the two terms in $O((M + N)K^2 + |\mathcal{R}|K)$.

6.4 Comparison against Spark MLlib's ALS

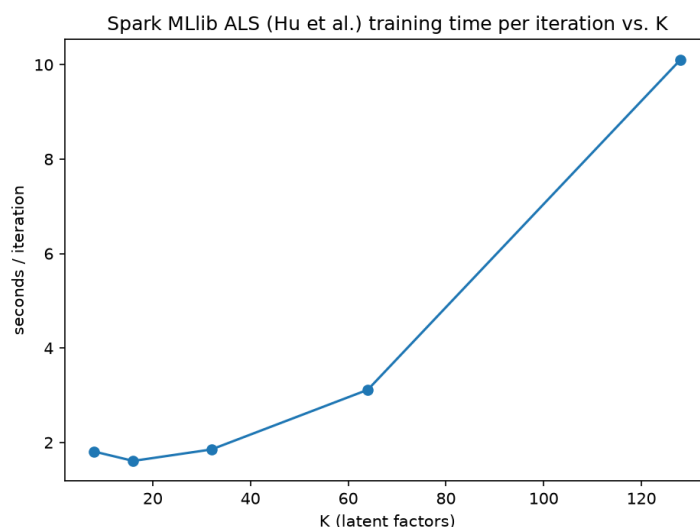


Figure 6.4: Spark MLlib's ALS(`implicitPrefs=True`) training time per iteration versus K , same dataset.

As a reference point independent of this project's own implementation, Table 6.4 repeats the K -sweep from Section 6.1 using Spark MLlib's built-in ALS(`implicitPrefs=True`), which implements Hu et al.'s uniform-weight whole-data objective with a production-grade, JVM-native vector-wise solver (`src/baselines.py`).

K	seconds/iteration	HR@10	NDCG@10
8	1.81	0.508	0.315
16	1.61	0.545	0.345
32	1.85	0.562	0.364
64	3.12	0.567	0.374
128	10.10	0.557	0.370

Table 6.4: Spark MLlib ALS: training time and 10-iteration accuracy versus K .

MLlib's implementation is faster in absolute terms across the whole sweep. That is unsurprising, since it is a mature, JIT-compiled Scala/Breeze implementation running matrix inversion via a near- $O(K^{2.376})$ algorithm rather than a Python-loop-bound coordinate-descent update, and the paper itself notes that real implementations of vector-wise ALS never show the full $O(K^3)$ growth their asymptotic analysis predicts, for the same reason. One entry does not fit a clean growth curve: $K = 16$ (1.61 s) is marginally faster than $K = 8$ (1.81 s), which is not consistent with any monotonic cost model. At well under two seconds per iteration, these runs are short enough that fixed overhead, JVM warm-up and Spark job dispatch that does not scale with K at all, plausibly dominates whatever small, genuine K -dependent cost separates the two, so this single inversion is best read as noise rather than a real non-monotonicity in MLlib's cost. What matters for this report's argument is the *shape* of the curve, not the absolute numbers: MLlib's own timings grow $5.6\times$ from $K = 8$ to $K = 128$ ($16\times$ increase in K), similar in kind to eALS's $10.9\times$ growth over the same range. Both are sub-cubic,

both broadly consistent with the paper's claim that a well-implemented ALS variant does not actually need to pay the full $O(K^3)$ its naive complexity analysis suggests, and both a very different regime from what a textbook matrix-inversion implementation without any of these optimizations would show.

Chapter 7

Discussion

This chapter steps back from individual figures and tables to ask what the experiments in Chapters 5 and 6 actually established, where they fell short of the paper’s own results, and why.

7.1 What worked

The efficiency claim holds up, in a setting the paper never tested. The paper’s argument for eALS is that it avoids the $O(K^3)$ matrix inversion of classical whole-data ALS while staying exactly as parallelizable. Both halves were checked directly: Section 6.1 measured $10.9\times$ growth in training time for a $16\times$ growth in K , nowhere close to the $4096\times$ a genuinely cubic solver would need. Section 3.2’s independence argument, that the parallel and sequential update rules are mathematically identical and not merely “close enough”, was validated against a synthetic dataset (Section 3.3.2) with no special-casing for the distributed execution path.

Exact parallelism is a real advantage over gradient-based alternatives. Every worker in the user-update sweep reads the same frozen Q and S^q and writes to a disjoint slice of P , so there is no analogue of the gradient-conflict problem that makes naively parallelized SGD either slow (if synchronized) or approximate (if not). This is the paper’s own argument, but building the actual PySpark version confirmed that it survives contact with a real distributed runtime rather than remaining a property of the equations alone.

The optimal missing-data weight $c_0 = 512$ transferred exactly. Given how differently this project’s dataset was curated compared to the paper’s 2016 snapshot (a third of the users, a fifth of the interactions, see Table 4.1), it was not obvious that an absolute-magnitude hyperparameter like c_0 would land on the same optimum. That it did (Figure 5.1) is a reassuring sign that the underlying mechanism, balancing observed-data loss against a much larger pool of weighted missing entries, behaves consistently across dataset scale, at least along this one axis.

7.2 What did not work as well, and why

Popularity-aware weighting’s advantage over uniform weighting was much smaller here than in the paper. The paper reports a statistically significant improvement from $\alpha = 0$ to $\alpha \approx 0.4$, confirmed with a paired t -test across repeated trials; this project found only a marginal, statistically untested difference (Section 5.1). Three factors plausibly contribute, though this project’s setup cannot fully separate them: the dataset is roughly a fifth the size

of the paper's, and popularity skew tends to be less extreme in smaller, more heavily-filtered samples; the sampled-negative evaluation protocol (Chapter 4) is a coarser ranking signal than the paper's full-catalog ranking and could plausibly wash out a small real effect; and each configuration here was trained once, with no repeated runs to average out initialization noise, unlike the paper's repeated-trial significance tests. Repeating every configuration and re-running full-catalog evaluation at scale were not practical within this project's resources, but both would be the direct way to determine whether the gap is a property of the dataset or of these measurement choices.

Parallelism stopped helping past four partitions, on this machine. Section 6.2 found the sweet spot at 4 partitions on a 4-core laptop, with 8 and 16 performing slightly worse. This is a finding about the hardware, not the algorithm: the update rule stays exactly as parallelizable at 16 partitions as at 4 (Section 3.2's argument does not depend on partition count), but Spark's per-task scheduling overhead does not shrink with the work each task has to do, so past a point adding partitions only adds overhead. On a real multi-node cluster the constraint shifts from per-task scheduling to inter-node transfer of the broadcast Q/S^q matrices, so this experiment's specific numbers would not transfer, though the underlying diminishing-returns trade-off almost certainly would. Confirming that on real cluster hardware was outside what this project could test.

The inner update loop is plain Python, not vectorized across users. Listing 3.1 loops over one user at a time inside each partition, even though a single user's update is only a handful of NumPy operations over a short neighbor list. That per-user Python overhead is the most likely reason MLlib's JVM-native ALS is 3-6 $\times$ faster in absolute terms across every K tested in Section 6.4, despite implementing a costlier theoretical algorithm. Batching several users' updates into fewer, larger NumPy calls would likely close much of that gap without touching the parallelization strategy or the underlying math at all. This was not attempted here, since closing an implementation-efficiency gap is a different question from validating the paper's algorithmic claims, which is what this project prioritized.

Scope: offline eALS only, not the paper's online extension. As flagged in Chapter 1, this project implements the paper's Algorithm 1 and its parallelization, not Algorithm 2 (the incremental online update) or the online evaluation protocol of the paper's Section 5.3. This matches the assignment brief, which asks specifically for an efficient, parallelizable MF-ALS algorithm, but it does mean this report cannot speak to the "online recommendation" half of the paper's title.

Two of these gaps, the smaller popularity-weighting effect and the non-matching absolute accuracy numbers, trace back to the same root cause: a differently-curated, roughly five-times-smaller dataset than the paper's, combined with a coarser sampled-negative evaluation used to keep the scalability study tractable. Both choices are stated where they are made (Chapter 4), and their combined effect is that this report's accuracy numbers are internally consistent with each other but should not be read as digit-for-digit comparable to the paper's own Table and Figure values.

Chapter 8

Conclusion

This project set out to answer a question the paper itself leaves open: does eALS's parallelizability, argued on paper, hold up when actually built in a distributed engine? The answer, based on the experiments in Chapters 5 and 6, is yes. The PySpark implementation reproduces the paper's element-wise update rule for term (Chapter 3), reaches the same optimal missing-data weight $c_0 = 512$ the paper reports for the Amazon dataset despite training on a differently-sized snapshot of it, converges to a stable solution within roughly ten iterations with no learning rate to tune, and scales in training time with $10.9\times$ growth for a $16\times$ increase in K , far below the $4096\times$ a cubic solver would need, and it lands in the same qualitative regime as Spark MLlib's own production-grade ALS.

Not everything reproduced cleanly. The popularity-aware weighting scheme's advantage over plain uniform weighting, a clearly significant effect in the paper, showed up here only as a small and statistically untested difference (Section 5.1), most plausibly because this project's dataset, filtered from a different and smaller snapshot of the same source, carries a less pronounced popularity signal than the one the paper trained on. The parallelism experiment's finding that four partitions beat eight or sixteen (Section 6.2) is a statement about single-machine task-scheduling overhead, not about the algorithm's parallelizability itself, which Section 3.2's independence argument guarantees regardless of partition count. And the project deliberately stops at the paper's offline Algorithm 1, leaving its online incremental-update extension (Algorithm 2) out of scope, in line with what the assignment brief asks for.

Building the implementation also surfaced a lesson that the paper's own complexity analysis does not make visible. At this project's scale, the gap between eALS and Spark MLlib's ALS was not decided by which algorithm does asymptotically less work, but by an engineering choice: broadcasting Q and S^q once per sweep instead of shuffling factor updates through Spark's DataFrame machinery, and a plain Python loop instead of a vectorized or compiled one (Section 7.2). A correct distributed algorithm and a fast distributed implementation of it are two different things to get right, and this project only fully closed the gap on the first.

The assignment brief really asks two questions: is eALS efficient, and is it parallelizable without approximation? Both held up under direct measurement, on real data, in a real distributed engine, which is the one thing the paper's own single-threaded benchmarks could not show.

Bibliography

- Robin Devooght, Nicolas Kourtellis, and Amin Mantrach. Dynamic matrix factorization with priors on unknown values. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '15)*, pages 189–198, 2015.
- Xiangnan He, Hanwang Zhang, Min-Yen Kan, and Tat-Seng Chua. Fast matrix factorization for online recommendation with implicit feedback. In *Proceedings of the 39th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '16)*, pages 549–558. ACM, 2016. doi: 10.1145/2911451.2911489.
- Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web (WWW '17)*, pages 173–182, 2017.
- Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative filtering for implicit feedback datasets. In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM '08)*, pages 263–272. IEEE, 2008.
- Julian McAuley. Amazon product data: Movies and tv ratings. https://snap.stanford.edu/data/amazon/productGraph/categoryFiles/ratings_Movies_and_TV.csv, 2014. Stanford Network Analysis Project (SNAP).
- Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. Bpr: Bayesian personalized ranking from implicit feedback. In *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence (UAI '09)*, pages 452–461, 2009.

Appendix A

Code and Reproducibility

The complete source code, including the notebook, the raw and processed data (tracked with Git LFS), and this report's own $\text{\LaTeX}$ source, is available at https://github.com/sepanta007/Matrix_Factorization. The listings below reproduce the core modules directly for convenience, exactly as used to produce the results in this report.

A.1 Repository layout

The project repository is organized as follows; every path below is relative to the project root.

```
src/
  eals.py           eALS core algorithm (Algorithm 1), PySpark RDD-based
  baselines.py      Spark MLlib ALS reference baseline
  evaluate.py       HR@K / NDCG@K leave-one-out evaluation
  preprocess.py     raw Amazon reviews -> filtered train/test parquet
  run_experiments.py  weighting / convergence / scalability experiments
  plot_results.py   turns results/*.csv into the figures used in this
                    report
tests/
  test_eals_synthetic.py  correctness check on a small synthetic dataset
notebooks/
  eals_demo.ipynb         small-data, end-to-end, fully executed
                        notebook
data/
  raw/                   downloaded Amazon ratings CSV
  processed/             filtered/reindexed train & test parquet
results/
  *.csv                  experiment outputs
  figures/*.png          figures used throughout this report
```

Listing A.1: Repository layout.

A.2 Reproducing the experiments

```
1 python -m venv .venv && source .venv/bin/activate
2 pip install pyspark numpy pandas scipy matplotlib jupyter nbformat pyarrow
   setuptools
3
4 # 1. Download and filter the dataset
5 curl -o data/raw/ratings_Movies_and_TV.csv \
```

```

6   https://snap.stanford.edu/data/amazon/productGraph/categoryFiles/
   ratings_Movies_and_TV.csv
7   python src/preprocess.py --raw data/raw/ratings_Movies_and_TV.csv \
8       --out data/processed --min-interactions 10
9
10  # 2. Run the weighting / convergence / scalability study
11  python src/run_experiments.py --data data/processed --out results
12  python src/plot_results.py --results results
13
14  # 3. Correctness check on synthetic data
15  python tests/test_eals_synthetic.py
16
17  # 4. Execute the small-data demonstration notebook end to end
18  jupyter nbconvert --to notebook --execute --inplace notebooks/eals_demo.
   ipynb

```

Listing A.2: Commands to reproduce every experiment in this report from scratch.

A.3 The small-data demonstration notebook

`notebooks/eals_demo.ipynb` exercises the entire pipeline, from data generation through training, evaluation, and inspection of the learned model, on data small enough to read and verify by hand. It does not depend on the large Amazon download being present (it is skipped gracefully, with a printed message, if `data/processed/` does not exist). Each code cell carries short comments explaining what it reads and what it produces where that is not obvious from the code itself. The notebook was executed top to bottom with `jupyter nbconvert -execute` immediately before this report was finalized, with zero errors in any cell; its structure is:

1. **Setup.** *Input:* none. *Output:* a local Spark session, with `src/eals.py` registered via `sc.addPyFile` so the closed-form update functions are importable on Spark's (local) executors, not just the driver.
2. **Synthetic dataset.** *Input:* none (a deterministic generator with a fixed seed). *Output:* `train_pairs/` `test_pairs` for three disjoint 15-user/15-item communities (45 users, 45 items, 225 training interactions, 45 held-out test interactions), small enough that a reader can manually verify the block structure the model is expected to recover.
3. **Training.** *Input:* the synthetic training pairs. *Output:* a fitted EALS model (`model.P`, `model.Q`) and its per-iteration loss history, plotted to show monotonic-ish convergence (loss drops from 145.7 to 83.8 over 20 iterations).
4. **Inspecting recommendations.** *Input:* the trained model and a probed user id. *Output:* that user's top-10 recommended items, printed with their true community label. All ten come from the user's own community, the expected result for a correctly trained model on this synthetic structure.
5. **Offline evaluation.** *Input:* the trained model and the held-out test pairs. *Output:* `HR@10/NDCG@10` computed two ways: `evaluate_full`, the paper's exact protocol (ranking against the whole 45-item catalog, tractable here because the catalog is tiny), and `evaluate_sampled`, the relaxed protocol used for the large-scale experiments in this report.

6. **Real-data subsample (optional).** *Input:* a random 300-user subsample of the preprocessed train/test parquet files under data/processed/, if present. *Output:* the same training/evaluation pipeline run on real (if reduced) Amazon interaction data, producing HR@10/NDCG@10 for that subsample in well under a minute.

A.4 Full source listings

The complete, unabridged source of every module described in Chapter 3 is reproduced below, exactly as used to produce the results in this report.

A.4.1 src/eals.py: the eALS algorithm

```

1  # eALS (element-wise ALS), Algorithm 1 of He et al. SIGIR'16, implemented
   in
2  # PySpark: the  $S^q/S^p$  caches are computed once per sweep and broadcast,
   and
3  # the per-user / per-item closed-form updates run in parallel via
   mapPartitions.
4  import functools
5  from dataclasses import dataclass
6  from typing import Dict, List, Tuple
7
8  import numpy as np
9
10 # force prints to flush immediately even when redirected to a log file (
   nohup, '>')
11 print = functools.partial(print, flush=True)
12
13
14 @dataclass
15 class EALSConfig:
16     n_factors: int = 32
17     c0: float = 512.0          # overall weight of missing data, Eq. (8)
18     alpha: float = 0.4        # popularity exponent, Eq. (8); alpha=0 ->
   uniform weighting (Hu et al. 2008)
19     reg: float = 0.01         # lambda, L2 regularization
20     w_obs: float = 1.0        # weight  $w_{ui}$  of observed (positive)
   entries
21     n_iters: int = 10
22     seed: int = 0
23     n_partitions: int = 8
24
25
26 def item_confidence(item_pos_counts: np.ndarray, cfg: EALSConfig) -> np.
   ndarray:
27     """ $c_i = c0 * f_i^{-\alpha} / \sum_j f_j^{-\alpha}$ , Eq. (8).  $f_i$  is item  $i$ 's
   share of interactions."""
28     freq = item_pos_counts / item_pos_counts.sum()
29     weighted = np.power(freq, cfg.alpha)
30     return cfg.c0 * weighted / weighted.sum()
31
32
33 def init_factors(n: int, k: int, seed: int) -> np.ndarray:
34     rng = np.random.RandomState(seed)
35     return (0.01 * rng.standard_normal(size=(n, k))).astype(np.float64)
36
37

```

```

38 def factor_cache(factors: np.ndarray, conf: np.ndarray = None) -> np.
   ndarray:
39     """S = sum_i conf_i * f_i f_i^T. S^q uses conf=item_confidence (Eq.
       10, 11); S^p uses conf=None, i.e. weight 1 -- the item's own c_i is
       applied separately in Eq. (13), see _update_items_chunk."""
40     if conf is None:
41         return factors.T @ factors
42     return (factors * conf[:, None]).T @ factors
43
44
45 def _update_users_chunk(
46     ids_chunk: np.ndarray,
47     user_ratings: Dict[int, np.ndarray],
48     P: np.ndarray,
49     Q: np.ndarray,
50     item_conf: np.ndarray,    # c_i, one value per item -- indexed by
   neighbor (Eq. 12)
51     Sq: np.ndarray,
52     reg: float,
53     w_obs: float,
54 ) -> np.ndarray:
55     """Eq. (12): closed-form update of p_u for every user in 'ids_chunk',
       iterating f=1..K."""
56     K = P.shape[1]
57     out = np.empty((len(ids_chunk), K), dtype=np.float64)
58     for row, u in enumerate(ids_chunk):
59         neighbors = user_ratings.get(u)
60         pu = P[u].copy()
61         if neighbors is None or len(neighbors) == 0:
62             out[row] = pu
63             continue
64         Qu = Q[neighbors]                # |R_u| x K
65         cu = item_conf[neighbors]        # |R_u|, c_i for each item u
   interacted with
66         rhat = Qu @ pu
67         for f in range(K):
68             rhat_f = rhat - pu[f] * Qu[:, f]
69             numer = np.sum((w_obs - (w_obs - cu) * rhat_f) * Qu[:, f]) - (
   pu @ Sq[:, f] - pu[f] * Sq[f, f])
70             denom = np.sum((w_obs - cu) * Qu[:, f] ** 2) + Sq[f, f] + reg
71             new_val = numer / denom if denom > 1e-12 else 0.0
72             rhat = rhat_f + new_val * Qu[:, f]
73             pu[f] = new_val
74         out[row] = pu
75     return out
76
77
78 def _update_items_chunk(
79     ids_chunk: np.ndarray,
80     item_ratings: Dict[int, np.ndarray],
81     Q: np.ndarray,
82     P: np.ndarray,
83     item_conf: np.ndarray,    # c_i for the items in this chunk -- one
   scalar per row (Eq. 13)
84     Sp: np.ndarray,
85     reg: float,
86     w_obs: float,
87 ) -> np.ndarray:
88     """Eq. (13): closed-form update of q_if for every item in 'ids_chunk',
       iterating f=1..K."""
89     K = Q.shape[1]

```

```

90     out = np.empty((len(ids_chunk), K), dtype=np.float64)
91     for row, i in enumerate(ids_chunk):
92         neighbors = item_ratings.get(i)
93         qi = Q[i].copy()
94         ci = item_conf[i]
95         if neighbors is None or len(neighbors) == 0:
96             out[row] = qi
97             continue
98         Pu = P[neighbors] # |Ri| x K
99         rhat = Pu @ qi
100         for f in range(K):
101             rhat_f = rhat - qi[f] * Pu[:, f]
102             numer = np.sum((w_obs - (w_obs - ci) * rhat_f) * Pu[:, f]) -
ci * (qi @ Sp[:, f] - qi[f] * Sp[f, f])
103             denom = np.sum((w_obs - ci) * Pu[:, f] ** 2) + ci * Sp[f, f] +
reg
104             new_val = numer / denom if denom > 1e-12 else 0.0
105             rhat = rhat_f + new_val * Pu[:, f]
106             qi[f] = new_val
107         out[row] = qi
108     return out
109
110
111 class EALS:
112     """Trains user/item latent factors with eALS on a Spark session.
113
114     'train_pairs' is a list of (user, item) positive (implicit)
115     interactions,
116     users/items already re-indexed to [0, n_users) / [0, n_items).
117     """
118     def __init__(self, cfg: EALSConfig):
119         self.cfg = cfg
120         self.P: np.ndarray = None
121         self.Q: np.ndarray = None
122         self.item_conf: np.ndarray = None
123         self.history: List[dict] = []
124
125     def fit(self, sc, n_users: int, n_items: int, train_pairs: List[Tuple[
int, int]], verbose: bool = True,
126             eval_fn=None, eval_every: int = 1):
127         """
128         eval_fn, if given, is called as eval_fn(model, iteration) after
129         every
130         'eval_every'-th iteration; its return value (a dict) is merged
131         into
132         self.history. Used to trace HR@K/NDCG@K vs. iteration count (
133         Figure 2
134         of the paper) without retraining from scratch at each checkpoint.
135         """
136         cfg = self.cfg
137
138         # adjacency lists (Ru, Ri) built once and reused every iteration
139         user_ratings: Dict[int, np.ndarray] = {}
140         item_ratings: Dict[int, np.ndarray] = {}
141         item_pos_counts = np.ones(n_items) # Laplace smoothing so unseen
142         items keep ci > 0
143         _tmp_u: Dict[int, list] = {}
144         _tmp_i: Dict[int, list] = {}
145         for u, i in train_pairs:
146             _tmp_u.setdefault(u, []).append(i)

```

```

143         _tmp_i.setdefault(i, []).append(u)
144         item_pos_counts[i] += 1
145         user_ratings = {u: np.array(v, dtype=np.int64) for u, v in _tmp_u.
items()}
146         item_ratings = {i: np.array(v, dtype=np.int64) for i, v in _tmp_i.
items()}
147
148         self.item_conf = item_confidence(item_pos_counts, cfg)
149         self.P = init_factors(n_users, cfg.n_factors, cfg.seed)
150         self.Q = init_factors(n_items, cfg.n_factors, cfg.seed + 1)
151
152         #glom() collapses each partition into one array, so the closed-
form
153         # update below runs once per partition instead of once per row
154         user_chunks = sc.parallelize(np.arange(n_users), cfg.n_partitions)
.glom()
155         item_chunks = sc.parallelize(np.arange(n_items), cfg.n_partitions)
.glom()
156
157         ur_b = sc.broadcast(user_ratings)
158         ir_b = sc.broadcast(item_ratings)
159         ic_b = sc.broadcast(self.item_conf)
160
161         for it in range(cfg.n_iters):
162             # ---- update user factors: Algorithm 1, lines 4-11
-----
163             Sq = factor_cache(self.Q, self.item_conf)
164             Q_b, Sq_b, P_local = sc.broadcast(self.Q), sc.broadcast(Sq),
self.P
165
166             def upd_users(ids_chunk, ur=ur_b, Qb=Q_b, Sqb=Sq_b, icb=ic_b,
cfg=cfg, P=P_local):
167                 ids_chunk = np.asarray(ids_chunk)
168                 res = _update_users_chunk(ids_chunk, ur.value, P, Qb.value
, icb.value, Sqb.value, cfg.reg, cfg.w_obs)
169                 return list(zip(ids_chunk.tolist(), res.tolist()))
170
171             for uid, vec in user_chunks.flatMap(upd_users).collect():
172                 self.P[uid] = vec
173                 Q_b.unpersist(); Sq_b.unpersist()
174
175             # ---- update item factors: Algorithm 1, lines 12-19
-----
176             Sp = factor_cache(self.P)
177             P_b, Sp_b, Q_local = sc.broadcast(self.P), sc.broadcast(Sp),
self.Q
178
179             def upd_items(ids_chunk, ir=ir_b, Pb=P_b, Spb=Sp_b, icb=ic_b,
cfg=cfg, Q=Q_local):
180                 ids_chunk = np.asarray(ids_chunk)
181                 res = _update_items_chunk(ids_chunk, ir.value, Q, Pb.value
, icb.value, Spb.value, cfg.reg, cfg.w_obs)
182                 return list(zip(ids_chunk.tolist(), res.tolist()))
183
184             for iid, vec in item_chunks.flatMap(upd_items).collect():
185                 self.Q[iid] = vec
186                 P_b.unpersist(); Sp_b.unpersist()
187
188             # only keep a history entry if we actually logged something
this round
189             record = {"iter": it + 1}

```

```

190         if verbose:
191             record["loss"] = self.compute_loss(train_pairs)
192             print(f"[eALS] iter {it + 1}/{cfg.n_iters} loss={record['loss']:.4f}")
193             if eval_fn is not None and (it + 1) % eval_every == 0:
194                 record.update(eval_fn(self, it + 1))
195             if len(record) > 1:
196                 self.history.append(record)
197
198         ur_b.unpersist(); ir_b.unpersist(); ic_b.unpersist()
199         return self
200
201     def compute_loss(self, train_pairs) -> float:
202         """Objective L, Eq. (14), evaluated vectorized in NumPy on the driver."""
203         cfg = self.cfg
204         Sq = factor_cache(self.Q, self.item_conf)
205         missing_term = np.einsum("uk,kl,ul->u", self.P, Sq, self.P).sum()
206
207         users = np.fromiter((u for u, _ in train_pairs), dtype=np.int64,
208                             count=len(train_pairs))
209         items = np.fromiter((i for _, i in train_pairs), dtype=np.int64,
210                             count=len(train_pairs))
211         rhat = np.einsum("nk,nk->n", self.P[users], self.Q[items])
212         obs_term = cfg.w_obs * np.sum((1.0 - rhat) ** 2)
213         correction = np.sum(self.item_conf[items] * rhat ** 2)
214
215         reg_term = cfg.reg * (np.sum(self.P ** 2) + np.sum(self.Q ** 2))
216         return float(obs_term + (missing_term - correction) + reg_term)
217
218     def predict(self, u: int, i: int) -> float:
219         return float(self.P[u] @ self.Q[i])
220
221     def score_all_items(self, u: int) -> np.ndarray:
222         return self.Q @ self.P[u]

```

A.4.2 src/evaluate.py: HR@K / NDCG@K evaluation

```

1  # HR@k / NDCG@k for the leave-one-out protocol. evaluate_full ranks
2  # against
3  # the whole catalog (paper's exact protocol); evaluate_sampled ranks
4  # against
5  # n_negatives random items instead, used for the large-scale experiments.
6  from typing import Dict, List, Tuple
7
8  import numpy as np
9
10 def _hr_ndcg_at_k(rank: int, k: int) -> Tuple[float, float]:
11     # rank is 0-indexed, so a hit at the very top scores log2(2) = 1
12     if rank < 0 or rank >= k:
13         return 0.0, 0.0
14     return 1.0, 1.0 / np.log2(rank + 2)
15
16 def evaluate_sampled(
17     P: np.ndarray,
18     Q: np.ndarray,

```

```

19     test_pairs: List[Tuple[int, int]],
20     train_user_items: Dict[int, set],
21     k: int = 10,
22     n_negatives: int = 99,
23     seed: int = 0,
24 ) -> Dict[str, float]:
25     """HR@k / NDCG@k, ranking each held-out positive against 'n_negatives'
        sampled negatives."""
26     rng = np.random.RandomState(seed)
27     n_items = Q.shape[0]
28     hrs, ndcgs = [], []
29     for u, pos_item in test_pairs:
30         seen = train_user_items.get(u, set())
31         negatives = []
32         while len(negatives) < n_negatives: # rejection sampling, fine at
            this n_items
33             cand = rng.randint(0, n_items)
34             if cand != pos_item and cand not in seen:
35                 negatives.append(cand)
36         candidates = np.array([pos_item] + negatives)
37         scores = Q[candidates] @ P[u]
38         rank = int(np.sum(scores > scores[0])) # how many candidates
            score above the positive
39         hr, ndcg = _hr_ndcg_at_k(rank, k)
40         hrs.append(hr)
41         ndcgs.append(ndcg)
42     return {"HR%d" % k: float(np.mean(hrs)), "NDCG%d" % k: float(np.mean(
        ndcgs)), "n_test": len(test_pairs)}
43
44
45 def evaluate_full(
46     P: np.ndarray,
47     Q: np.ndarray,
48     test_pairs: List[Tuple[int, int]],
49     train_user_items: Dict[int, set],
50     k: int = 100,
51 ) -> Dict[str, float]:
52     """Exact paper protocol: rank the held-out item against the *entire*
        catalog
        (excluding items already seen in training). Only tractable for small M
        , N."""
53     hrs, ndcgs = [], []
54     for u, pos_item in test_pairs:
55         scores = Q @ P[u]
56         seen = train_user_items.get(u, set())
57         if seen:
58             scores = scores.copy()
59             scores[list(seen)] = -np.inf
60         rank = int(np.sum(scores > scores[pos_item]))
61         hr, ndcg = _hr_ndcg_at_k(rank, k)
62         hrs.append(hr)
63         ndcgs.append(ndcg)
64     return {"HR%d" % k: float(np.mean(hrs)), "NDCG%d" % k: float(np.mean(
        ndcgs)), "n_test": len(test_pairs)}

```

A.4.3 src/baselines.py: Spark MLlib ALS reference

```

1 # Spark MLlib's own implicit ALS, used as an external reference point.

```

```

2 from typing import List, Tuple
3
4 from pyspark.sql import SparkSession, functions as F
5 from pyspark.ml.recommendation import ALS as SparkALS
6
7
8 def fit_spark_mllib_als(
9     spark: SparkSession,
10     train_pairs: List[Tuple[int, int]],
11     n_factors: int = 32,
12     reg: float = 0.01,
13     n_iters: int = 10,
14     alpha: float = 1.0, # Mllib's own confidence scaling, unrelated to
        eALS's popularity alpha
15 ):
16     """Spark's own implicit-feedback ALS (Hu et al. 2008), for reference.
        """
17     df = spark.createDataFrame(train_pairs, ["user", "item"]).withColumn("
        rating", F.lit(1.0))
18     als = SparkALS(
19         userCol="user", itemCol="item", ratingCol="rating",
20         rank=n_factors, maxIter=n_iters, regParam=reg, alpha=alpha,
21         implicitPrefs=True, coldStartStrategy="drop", seed=0,
22     )
23     return als.fit(df)

```

A.4.4 src/preprocess.py: dataset filtering and splitting

```

1 # Filters the raw Amazon reviews down to a 10-core implicit-feedback
    dataset
2 # and writes the reindexed train/test parquet files used by every
    experiment.
3 import argparse
4 import functools
5 import shutil
6
7 from pyspark.sql import SparkSession, functions as F
8 from pyspark.sql.window import Window
9
10 # force prints to flush immediately even when redirected to a log file (
    nohup, '>')
11 print = functools.partial(print, flush=True)
12
13
14 def iterative_core_filter(spark, df, min_interactions, tmp_dir, max_rounds
    =50):
15     """Repeatedly drop users/items below the interaction threshold until
        stable (k-core filtering).
16     Each round is materialized to parquet and reloaded from disk rather
        than just '.cache()' -d:
17     caching alone keeps extending the logical query plan every round (
        dedup -> N joins), and Spark's
18     adaptive-execution plan-string logging on that ever-growing plan is
        what triggered the driver
19     OutOfMemoryError seen at round ~6. Writing to disk truncates the
        lineage every round."""
20     before = df.count()
21     converged = False

```

```

22     for rnd in range(max_rounds):
23         keep_users = df.groupBy("user_raw").count().filter(F.col("count")
24         >= min_interactions).select("user_raw")
25         keep_items = df.groupBy("item_raw").count().filter(F.col("count")
26         >= min_interactions).select("item_raw")
27         filtered = df.join(keep_users, "user_raw").join(keep_items, "
28         item_raw")
29
30         round_path = f"{tmp_dir}/round_{rnd}.parquet"
31         filtered.write.mode("overwrite").parquet(round_path)
32         df = spark.read.parquet(round_path)
33
34         after = df.count()
35         print(f"    core-filter round {rnd + 1}: {before} -> {after}
36         interactions")
37         if after == before:
38             converged = True
39             break
40         before = after
41         if not converged:
42             print(f"    WARNING: core-filter did not fully converge within {
43             max_rounds} rounds "
44             f"(every remaining user/item still had >= {min_interactions}
45             interactions in the last "
46             f"round is NOT guaranteed) -- increase max_rounds if you
47             need an exact k-core.")
48         else:
49             print(f"    core-filter converged: every remaining user and item has
50             >= {min_interactions} interactions.")
51         return df
52
53 def main(raw_path, out_dir, min_interactions, max_rounds=50):
54     spark = (
55         SparkSession.builder.appName("eals-preprocess")
56         .config("spark.sql.shuffle.partitions", "32")
57         .config("spark.driver.memory", "6g")
58         .getOrCreate()
59     )
60
61     raw = spark.read.csv(
62         raw_path,
63         schema="user_raw STRING, item_raw STRING, rating DOUBLE, ts LONG",
64     )
65
66     tmp_dir = f"{out_dir}/_tmp_corefilter"
67     shutil.rmtree(tmp_dir, ignore_errors=True)
68
69     # Implicit feedback: any review/rating becomes a positive interaction
70     # (rui = 1).
71     # Duplicate (user, item) pairs collapse to their most recent timestamp
72     .
73     dedup_path = f"{tmp_dir}/dedup.parquet"
74     raw.groupBy("user_raw", "item_raw").agg(F.max("ts").alias("ts")).write
75     .mode("overwrite").parquet(dedup_path)
76     interactions = spark.read.parquet(dedup_path)
77     print("raw deduped interactions:", interactions.count())
78
79     interactions = iterative_core_filter(spark, interactions,
80     min_interactions, tmp_dir, max_rounds=max_rounds)

```



```

71  # row_number() starts at 1, subtract 1 so ids land in [0, n) like eals
72  .py expects
73  user_index = (
74      interactions.select("user_raw").distinct()
75      .withColumn("user", F.row_number().over(Window.orderBy("user_raw")
76      ) - 1)
77  )
78  item_index = (
79      interactions.select("item_raw").distinct()
80      .withColumn("item", F.row_number().over(Window.orderBy("item_raw")
81      ) - 1)
82  )
83  interactions = (
84      interactions.join(user_index, "user_raw")
85      .join(item_index, "item_raw")
86      .select("user", "item", "ts")
87  )
88
89  # Offline / leave-one-out protocol (Section 5.1): each user's
90  chronologically
91  # latest interaction is held out for testing, the rest is training
92  data.
93  w = Window.partitionBy("user").orderBy(F.col("ts").desc())
94  ranked = interactions.withColumn("rank", F.row_number().over(w))
95  test = ranked.filter(F.col("rank") == 1).select("user", "item")
96  train = ranked.filter(F.col("rank") > 1).select("user", "item")
97
98  interactions.select("user", "item", "ts").write.mode("overwrite").
99  parquet(f"{out_dir}/interactions.parquet")
100  train.write.mode("overwrite").parquet(f"{out_dir}/train.parquet")
101  test.write.mode("overwrite").parquet(f"{out_dir}/test.parquet")
102  user_index.write.mode("overwrite").parquet(f"{out_dir}/user_index.
103  parquet")
104  item_index.write.mode("overwrite").parquet(f"{out_dir}/item_index.
105  parquet")
106
107  n_users = user_index.count()
108  n_items = item_index.count()
109  n_train = train.count()
110  n_test = test.count()
111  print(f"users={n_users} items={n_items} train={n_train} test={n_test}")
112
113  f"sparsity={1 - (n_train + n_test) / (n_users * n_items):.4%}")
114
115  spark.stop()
116  shutil.rmtree(tmp_dir, ignore_errors=True)
117
118  if __name__ == "__main__":
119      parser = argparse.ArgumentParser()
120      parser.add_argument("--raw", default="data/raw/ratings_Movies_and_TV.
121      csv")
122      parser.add_argument("--out", default="data/processed")
123      parser.add_argument("--min-interactions", type=int, default=10)
124      args = parser.parse_args()
125      main(args.raw, args.out, args.min_interactions)

```

A.4.5 src/run_experiments.py: experiment driver

```

1  # Runs the weighting sweep, the eALS-vs-uniform convergence comparison,
    and
2  # the scalability study, each writing a CSV to results/.
3  import argparse
4  import functools
5  import os
6  import time
7  from typing import Dict, List
8
9  import numpy as np
10 from pyspark.sql import SparkSession
11
12 # stdout is block-buffered when redirected to a file/log (nohup, '>'), so
    plain
13 # print() calls sit unseen in a buffer until it fills or the process exits
    --
14 # making a long background run look stuck. Force a flush on every print
    instead.
15 print = functools.partial(print, flush=True)
16
17 from eals import EALS, EALSConfig
18 from baselines import fit_spark_mllib_als
19 from evaluate import evaluate_sampled
20
21
22 def load_data(spark, data_dir):
23     train_df = spark.read.parquet(f"{data_dir}/train.parquet")
24     test_df = spark.read.parquet(f"{data_dir}/test.parquet")
25     n_users = spark.read.parquet(f"{data_dir}/user_index.parquet").count()
26     n_items = spark.read.parquet(f"{data_dir}/item_index.parquet").count()
27
28     train_pairs = [(r["user"], r["item"]) for r in train_df.collect()]
29     test_pairs = [(r["user"], r["item"]) for r in test_df.collect()]
30
31     train_user_items: Dict[int, set] = {}
32     for u, i in train_pairs:
33         train_user_items.setdefault(u, set()).add(i)
34
35     return n_users, n_items, train_pairs, test_pairs, train_user_items
36
37
38 def weighting_study(sc, n_users, n_items, train_pairs, test_pairs,
    train_user_items, out_dir):
39     """Grid over c0 (alpha=0) then over alpha (c0 fixed at the best value)
        -- mirrors Figure 1."""
40     rows = []
41     for c0 in [8, 32, 128, 512, 2048]:
42         cfg = EALSConfig(n_factors=32, c0=c0, alpha=0.0, reg=0.01, n_iters
            =10, n_partitions=8)
43         model = EALS(cfg).fit(sc, n_users, n_items, train_pairs, verbose=
            False)
44         metrics = evaluate_sampled(model.P, model.Q, test_pairs,
            train_user_items, k=10)
45         rows.append({"sweep": "c0", "c0": c0, "alpha": 0.0, **metrics})
46         print(rows[-1])
47
48     best_c0 = max([r for r in rows if r["sweep"] == "c0"], key=lambda r: r
        ["HR@10"])["c0"]

```

```

49     for alpha in [0.0, 0.2, 0.4, 0.5, 0.6, 0.8]:
50         cfg = EALSConfig(n_factors=32, c0=best_c0, alpha=alpha, reg=0.01,
n_iters=10, n_partitions=8)
51         model = EALS(cfg).fit(sc, n_users, n_items, train_pairs, verbose=
False)
52         metrics = evaluate_sampled(model.P, model.Q, test_pairs,
train_user_items, k=10)
53         rows.append({"sweep": "alpha", "c0": best_c0, "alpha": alpha, **
metrics})
54         print(rows[-1])
55
56     _write_csv(rows, f"{out_dir}/weighting_study.csv")
57     return best_c0
58
59
60 def method_comparison(spark, sc, n_users, n_items, train_pairs, test_pairs
, train_user_items, out_dir, best_c0):
61     """Convergence (HR@10/NDCG@10 vs. iteration) for popularity-aware vs.
uniform-weight eALS -- mirrors Figure 2."""
62     rows = []
63     n_iters = 15
64
65     def make_eval_fn(name, t_start):
66         # runs every iteration, so elapsed_seconds here bundles eval time
in
67         # with training time -- not directly comparable to
scalability_study's numbers
68         def eval_fn(model, it):
69             metrics = evaluate_sampled(model.P, model.Q, test_pairs,
train_user_items, k=10)
70             row = {"method": name, "iter": it, "elapsed_seconds": time.
time() - t_start[0], **metrics}
71             rows.append(row)
72             return {}
73         return eval_fn
74
75     for name, alpha in [("eALS (popularity-aware)", 0.4), ("eALS (uniform
alpha=0)", 0.0)]:
76         cfg = EALSConfig(n_factors=32, c0=best_c0, alpha=alpha, reg=0.01,
n_iters=n_iters, n_partitions=8)
77         t_start = [time.time()]
78         EALS(cfg).fit(sc, n_users, n_items, train_pairs, verbose=False,
eval_fn=make_eval_fn(name, t_start))
79         print(name, rows[-1])
80
81     _write_csv(rows, f"{out_dir}/method_comparison.csv")
82
83
84 def scalability_study(sc, n_users, n_items, train_pairs, test_pairs,
train_user_items, out_dir, best_c0):
85     rows = []
86     # (a) training time per iteration vs. number of latent factors K
87     for K in [8, 16, 32, 64, 128]:
88         cfg = EALSConfig(n_factors=K, c0=best_c0, alpha=0.4, reg=0.01,
n_iters=3, n_partitions=8)
89         t0 = time.time()
90         model = EALS(cfg).fit(sc, n_users, n_items, train_pairs, verbose=
False)
91         elapsed = (time.time() - t0) / cfg.n_iters
92         metrics = evaluate_sampled(model.P, model.Q, test_pairs,
train_user_items, k=10)

```

```

93     rows.append({"axis": "K", "K": K, "n_partitions": 8, "
seconds_per_iter": elapsed, **metrics})
94     print(rows[-1])
95
96     # (b) training time per iteration vs. number of Spark partitions (
parallelism)
97     for P_ in [1, 2, 4, 8, 16]:
98         cfg = EALSConfig(n_factors=32, c0=best_c0, alpha=0.4, reg=0.01,
n_iters=3, n_partitions=P_)
99         t0 = time.time()
100        model = EALS(cfg).fit(sc, n_users, n_items, train_pairs, verbose=
False)
101        elapsed = (time.time() - t0) / cfg.n_iters
102        rows.append({"axis": "n_partitions", "K": 32, "n_partitions": P_,
"seconds_per_iter": elapsed})
103        print(rows[-1])
104
105        # (c) training time per iteration vs. training-set size (data
scalability)
106        rng = np.random.RandomState(0)
107        for frac in [0.1, 0.25, 0.5, 0.75, 1.0]:
108            idx = rng.choice(len(train_pairs), size=int(len(train_pairs) *
frac), replace=False)
109            sub_pairs = [train_pairs[i] for i in idx]
110            cfg = EALSConfig(n_factors=32, c0=best_c0, alpha=0.4, reg=0.01,
n_iters=3, n_partitions=8)
111            t0 = time.time()
112            EALS(cfg).fit(sc, n_users, n_items, sub_pairs, verbose=False)
113            elapsed = (time.time() - t0) / cfg.n_iters
114            rows.append({"axis": "data_frac", "K": 32, "n_partitions": 8, "
data_frac": frac,
115                        "n_train": len(sub_pairs), "seconds_per_iter":
elapsed})
116            print(rows[-1])
117
118        _write_csv(rows, f"{out_dir}/scalability_study.csv")
119
120
121 def spark_mllib_reference(spark, train_pairs, test_pairs, train_user_items
, n_users, n_items, out_dir):
122     rows = []
123     for K in [8, 16, 32, 64, 128]:
124         t0 = time.time()
125         model = fit_spark_mllib_als(spark, train_pairs, n_factors=K, reg
=0.01, n_iters=10)
126         elapsed = (time.time() - t0) / 10
127         # Mllib hands back factors as a DataFrame; pull them into the same
# dense P/Q layout evaluate_sampled expects
128         P = np.zeros((n_users, K)); Q = np.zeros((n_items, K))
129         for r in model.userFactors.collect():
130             P[r["id"]] = r["features"]
131         for r in model.itemFactors.collect():
132             Q[r["id"]] = r["features"]
133         metrics = evaluate_sampled(P, Q, test_pairs, train_user_items, k
=10)
134         rows.append({"K": K, "seconds_per_iter": elapsed, **metrics})
135         print(rows[-1])
136         _write_csv(rows, f"{out_dir}/spark_mllib_als.csv")
137
138
139
140 def _write_csv(rows: List[dict], path: str):

```

```

141     if not rows:
142         return
143     import csv
144     keys = sorted({k for r in rows for k in r.keys()})
145     with open(path, "w", newline="") as f:
146         writer = csv.DictWriter(f, fieldnames=keys)
147         writer.writeheader()
148         writer.writerows(rows)
149     print(f"wrote {path}")
150
151
152 def main():
153     parser = argparse.ArgumentParser()
154     parser.add_argument("--data", default="data/processed")
155     parser.add_argument("--out", default="results")
156     parser.add_argument("--skip-weighting", action="store_true")
157     parser.add_argument("--skip-mllib", action="store_true")
158     args = parser.parse_args()
159     os.makedirs(args.out, exist_ok=True)
160
161     spark = SparkSession.builder.appName("eals-experiments").config("spark
162     .driver.memory", "6g").getOrCreate()
163     sc = spark.sparkContext
164     sc.setLogLevel("WARN")
165     sc.addPyFile("src/eals.py")
166
167     n_users, n_items, train_pairs, test_pairs, train_user_items =
168     load_data(spark, args.data)
169     print(f"n_users={n_users} n_items={n_items} n_train={len(train_pairs)}
170     n_test={len(test_pairs)}")
171
172     best_c0 = 512.0
173     if not args.skip_weighting:
174         best_c0 = weighting_study(sc, n_users, n_items, train_pairs,
175         test_pairs, train_user_items, args.out)
176
177     method_comparison(spark, sc, n_users, n_items, train_pairs, test_pairs
178     , train_user_items, args.out, best_c0)
179     scalability_study(sc, n_users, n_items, train_pairs, test_pairs,
180     train_user_items, args.out, best_c0)
181
182     if not args.skip_mllib:
183         spark_mllib_reference(spark, train_pairs, test_pairs,
184         train_user_items, n_users, n_items, args.out)
185
186     spark.stop()
187
188 if __name__ == "__main__":
189     main()

```

A.4.6 src/plot_results.py: figure generation

```

1  # Turns the CSVs produced by run_experiments.py into PNG figures under
2  results/figures/.
3  import argparse
4  import os

```

```

5 import matplotlib.pyplot as plt
6 import pandas as pd
7
8
9 def plot_weighting_study(path, out_dir):
10     if not os.path.exists(path):
11         return
12     df = pd.read_csv(path)
13
14     c0_df = df[df.sweep == "c0"].sort_values("c0")
15     fig, ax1 = plt.subplots()
16     ax1.plot(c0_df["c0"], c0_df["HR@10"], "o-", label="HR@10")
17     ax1.set_xscale("log", base=2)
18     ax1.set_xlabel("c0 (alpha=0)")
19     ax1.set_ylabel("HR@10")
20     ax2 = ax1.twinx()
21     ax2.plot(c0_df["c0"], c0_df["NDCG@10"], "s--", color="orange", label="NDCG@10")
22     ax2.set_ylabel("NDCG@10")
23     fig.legend(loc="upper left", bbox_to_anchor=(0.15, 0.88))
24     plt.title("Impact of c0 (missing-data weight) on eALS")
25     plt.tight_layout()
26     plt.savefig(f"{out_dir}/weighting_c0.png", dpi=150)
27     plt.close()
28
29     a_df = df[df.sweep == "alpha"].sort_values("alpha")
30     fig, ax1 = plt.subplots()
31     ax1.plot(a_df["alpha"], a_df["HR@10"], "o-", label="HR@10")
32     ax1.set_xlabel("alpha (popularity exponent)")
33     ax1.set_ylabel("HR@10")
34     ax2 = ax1.twinx()
35     ax2.plot(a_df["alpha"], a_df["NDCG@10"], "s--", color="orange", label="NDCG@10")
36     ax2.set_ylabel("NDCG@10")
37     fig.legend(loc="upper left", bbox_to_anchor=(0.15, 0.88))
38     plt.title("Impact of alpha (popularity exponent) on eALS")
39     plt.tight_layout()
40     plt.savefig(f"{out_dir}/weighting_alpha.png", dpi=150)
41     plt.close()
42
43
44 def plot_method_comparison(path, out_dir):
45     if not os.path.exists(path):
46         return
47     df = pd.read_csv(path)
48     for metric in ["HR@10", "NDCG@10"]:
49         plt.figure()
50         for name, g in df.groupby("method"):
51             g = g.sort_values("iter")
52             plt.plot(g["iter"], g[metric], marker="o", label=name)
53         plt.xlabel("iteration")
54         plt.ylabel(metric)
55         plt.title(f"Convergence: {metric} vs. iteration")
56         plt.legend()
57         plt.tight_layout()
58         fname = metric.replace("@", "_at_")
59         plt.savefig(f"{out_dir}/convergence_{fname}.png", dpi=150)
60         plt.close()
61
62
63 def plot_scalability(path, out_dir):

```

```

64     if not os.path.exists(path):
65         return
66     df = pd.read_csv(path)
67
68     k_df = df[df.axis == "K"].sort_values("K")
69     if len(k_df):
70         plt.figure()
71         plt.plot(k_df["K"], k_df["seconds_per_iter"], "o-")
72         plt.xlabel("K (latent factors)")
73         plt.ylabel("seconds / iteration")
74         plt.title("eALS training time per iteration vs. K")
75         plt.tight_layout()
76         plt.savefig(f"{out_dir}/scalability_K.png", dpi=150)
77         plt.close()
78
79     p_df = df[df.axis == "n_partitions"].sort_values("n_partitions")
80     if len(p_df):
81         plt.figure()
82         plt.plot(p_df["n_partitions"], p_df["seconds_per_iter"], "o-")
83         plt.xlabel("Spark partitions")
84         plt.ylabel("seconds / iteration")
85         plt.title("eALS training time per iteration vs. parallelism")
86         plt.tight_layout()
87         plt.savefig(f"{out_dir}/scalability_partitions.png", dpi=150)
88         plt.close()
89
90     d_df = df[df.axis == "data_frac"].sort_values("n_train")
91     if len(d_df):
92         plt.figure()
93         plt.plot(d_df["n_train"], d_df["seconds_per_iter"], "o-")
94         plt.xlabel("training interactions")
95         plt.ylabel("seconds / iteration")
96         plt.title("eALS training time per iteration vs. data size")
97         plt.tight_layout()
98         plt.savefig(f"{out_dir}/scalability_data_size.png", dpi=150)
99         plt.close()
100
101
102 def plot_mllib_reference(path, out_dir):
103     if not os.path.exists(path):
104         return
105     df = pd.read_csv(path).sort_values("K")
106     plt.figure()
107     plt.plot(df["K"], df["seconds_per_iter"], "o-")
108     plt.xlabel("K (latent factors)")
109     plt.ylabel("seconds / iteration")
110     plt.title("Spark MLlib ALS (Hu et al.) training time per iteration vs. K")
111     plt.tight_layout()
112     plt.savefig(f"{out_dir}/mllib_scalability_K.png", dpi=150)
113     plt.close()
114
115
116 def main():
117     parser = argparse.ArgumentParser()
118     parser.add_argument("--results", default="results")
119     args = parser.parse_args()
120     out_dir = f"{args.results}/figures"
121     os.makedirs(out_dir, exist_ok=True)
122
123     plot_weighting_study(f"{args.results}/weighting_study.csv", out_dir)

```

```

124     plot_method_comparison(f"{args.results}/method_comparison.csv",
125                           out_dir)
126     plot_scalability(f"{args.results}/scalability_study.csv", out_dir)
127     plot_mllib_reference(f"{args.results}/spark_mllib_als.csv", out_dir)
128     print(f"figures written to {out_dir}")
129
130 if __name__ == "__main__":
131     main()

```

A.4.7 tests/test_eals_synthetic.py: correctness check

```

1  # Two disjoint user/item communities; a correct model should rank each
2  # user's own community above the other one and the loss should decrease.
3  import sys
4  import os
5
6  sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "src"))
7
8  import numpy as np
9  from pyspark.sql import SparkSession
10
11 from eals import EALS, EALSConfig
12
13
14 def make_two_community_data(n_users=20, n_items=20, seed=0):
15     rng = np.random.RandomState(seed)
16     pairs = []
17     # community A: users 0-9 interact with items 0-9
18     for u in range(10):
19         items = rng.choice(10, size=5, replace=False)
20         pairs += [(u, int(i)) for i in items]
21     # community B: users 10-19 interact with items 10-19
22     for u in range(10, 20):
23         items = rng.choice(range(10, 20), size=5, replace=False)
24         pairs += [(u, int(i)) for i in items]
25     return pairs, n_users, n_items
26
27
28 def main():
29     spark = SparkSession.builder.master("local[4]").appName("eals-test").
30     getOrCreate()
31     sc = spark.sparkContext
32     sc.setLogLevel("WARN")
33     sc.addPyFile(os.path.join(os.path.dirname(__file__), "..", "src", "
34     eals.py"))
35
36     pairs, n_users, n_items = make_two_community_data()
37     cfg = EALSConfig(n_factors=8, c0=32.0, alpha=0.4, reg=0.05, n_iters
38     =15, n_partitions=4)
39     model = EALS(cfg).fit(sc, n_users, n_items, pairs)
40
41     losses = [h["loss"] for h in model.history]
42     assert losses[-1] < losses[0], f"loss did not decrease: {losses}"
43
44     scores_u0 = model.score_all_items(0)
45     within = scores_u0[:10].mean()
46     across = scores_u0[10:].mean()

```



```
44     print(f"user 0: within-community avg score={within:.4f} across-  
community avg score={across:.4f}")  
45     assert within > across, "eALS failed to separate the two communities"  
46  
47     print("OK: loss history:", [f"{l:.2f}" for l in losses])  
48     print("PASSED")  
49     spark.stop()  
50  
51  
52 if __name__ == "__main__":  
53     main()
```